第9章: 運用・保守・デプロイ — 日々の運用と本番リリース

> この章では、完成したアプリを **毎日動かし続ける（運用）・壊れたら直す（保守）・新しい版を本番サーバーへ送り出す（デプロイ）** ための、具体的な手順とコードを学びます。
>
> プログラムは「書いて終わり」ではありません。むしろ、書き終えてからの方が長い付き合いになります。ユーザーが毎日触り、データが溜まり、たまに障害が起きます。あなた（後任者）が一番よく使うのは、新機能を作るコードよりも、**バックアップを取る・データを戻す・ログを見る・サーバーを再起動する** といった「運用の操作」です。
>
> この章は、本アプリの **バックアップ／復元のソースコード（ `kosu/tasks.py` ）を1関数ずつ・1行ずつ**、その処理を起動する **非同期フロー（ `kosu/views/asynchronous_views.py` ）を1行ずつ**、設定の心臓部 **`hozen_another/settings.py` を1行ずつ**、そして本番サーバーへの入口 **`wsgi.py`** を読み解きます。さらに、本番が動いている **Azure（アジュール）** というクラウドの構成、デプロイ手順、障害が起きたときの対応フローまでを、辞書のように引ける密度で書きます。
>
> 長い章ですが、**困ったときに開く「運用マニュアル」** として手元に置いてください。

この章で学ぶこと

- **3つの環境（ローカル／ステージング／本番）** — なぜ環境を分けるのか、それぞれの役割
- **日常運用コマンド集** — サーバー起動・マイグレーション・テスト実行・キュー起動を1つずつ
- **バックアップ機能の全体像** —「Excelに書き出す → 一時保存 → ダウンロード → 自動削除」の流れ
- **`tasks.py` を1行ずつ** — `openpyxl`（オープンパイエクセル）でExcelを作る／`pandas`（パンダス）で重複を消す／DBへ流し込む処理
- **`asynchronous_views.py` を1行ずつ** — タスクID発行・別スレッド起動・状態ポーリング・ファイル配信と後始末
- **`settings.py` を1行ずつ** — `LOGGING`（ログ設定）・`Q_CLUSTER`・`DATABASES`・`STATIC`／WhiteNoise・`ALLOWED_HOSTS`・`CSRF_TRUSTED_ORIGINS`
- **本番アーキテクチャ** — Azure App Service + Gunicorn + WhiteNoise + Azure Database for PostgreSQL フレキシブルサーバー
- **`wsgi.py`** — Webサーバーとアプリをつなぐ「玄関」
- **デプロイ手順と本番マイグレーションの注意** — やってはいけないこと
- **HTTPステータスコード表・障害対応フロー・定期メンテ・PDF・Web版の再生成手順**

目次

1. 運用・保守・デプロイとは（全体像）
2. 3つの環境：ローカル・ステージング・本番

3. 日常運用コマンド集
4. テストの実行
5. バックアップ／復元機能の全体像
6. tasks.py 全解説①：工数バックアップ（openpyxl）
7. tasks.py 全解説②：予測データ出力（pandas）
8. tasks.py 全解説③：ロード（Excel→DB）と削除
9. tasks.py 全解説④：人員・班員・定義・その他のバックアップ
10. asynchronous_views.py 全解説：非同期フローとポーリング
11. settings.py 全解説：本番設定の心臓部
12. wsgi.py 全解説：Webサーバーとの玄関
13. 本番アーキテクチャ：Azure 構成図
14. デプロイ手順と本番マイグレーションの注意
15. HTTPステータスコード早見表
16. 障害対応フロー
17. 定期メンテナンスとドキュメントのPDF・Web版の再生成
18. トラブルシューティング
19. 演習問題
20. この章のまとめ

---

1. 運用・保守・デプロイとは（全体像）

まず、3つの言葉の意味をはっきりさせます。これらは混同されがちですが、別の作業です。

> **用語：運用（うんよう：operation）** 完成したシステムを「日々動かし続ける」こと。サーバーの起動状態を保つ、バックアップを定期的に取る、ログを監視する、ユーザーからの問い合わせに答える、といった日常作業全般。家でたとえると「毎日の掃除・ゴミ出し・電気代の支払い」です。
>
> **用語：保守（ほしゅ：maintenance）** 不具合を直したり、性能を改善したり、データを整理したりする「メンテナンス」作業。家でたとえると「雨漏りを直す・古くなった水道管を交換する」です。
>
> **用語：デプロイ（deploy：配置・展開）** 作ったプログラム（新しい版）を、本番のサーバーへ送り込んで実際に動かすこと。「リリース」「本番反映」とも言います。家でたとえると「リフォームした部屋を、実際に住める状態にして引き渡す」です。

1.1 本アプリの登場人物（おさらい）

運用の話に入る前に、このシステムを構成する部品をおさらいします。

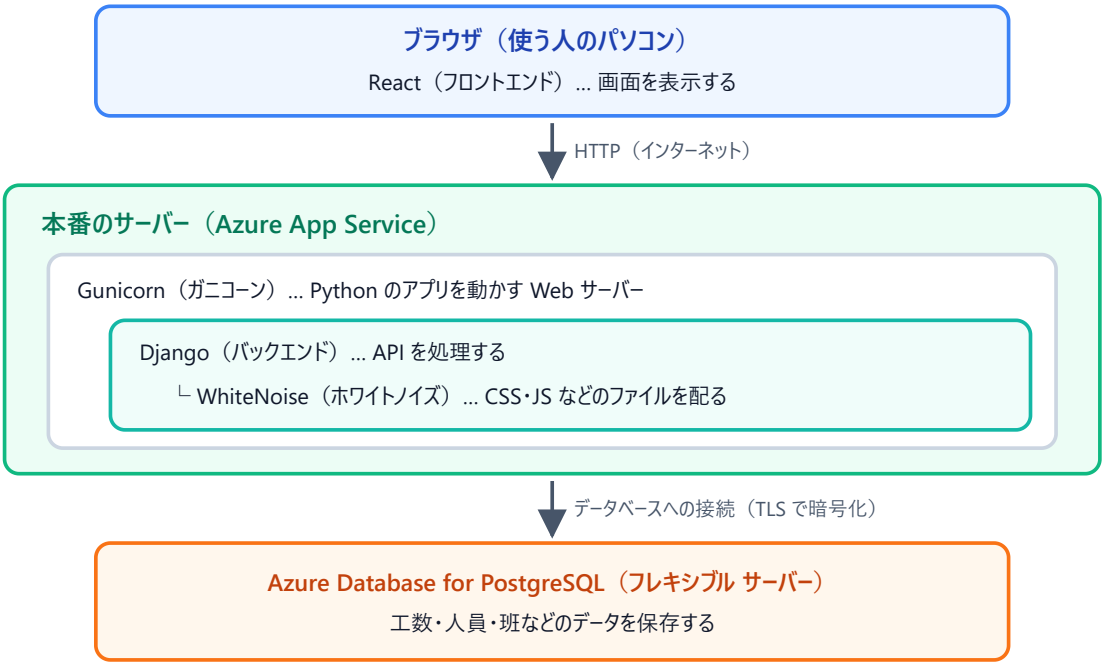


図9-1: 本番の構成。ブラウザ → Azure App Service（Gunicorn・Django・WhiteNoise）→ Azure Database for PostgreSQL

それぞれの部品の役割は §13 で詳しく説明します。この章では、この絵の **どの部品をどう操作するか** を1つずつ覚えていきます。

1.2 なぜ運用でコードを読むのか

> **なぜ？**「運用」というと、コードを読まずにボタンを押すだけと思うかもしれません。しかし本アプリのバックアップ機能は、画面のボタンを押すと **裏でPythonコードが走り**、Excelファイルを作っています。ボタンを押して「失敗しました」と出たとき、原因を突き止めるには、その裏で何が起きているかを知る必要があります。だからこの章では、運用機能の **実コードを丸ごと読む** のです。

---

2. 3つの環境：ローカル・ステージング・本番

プロのソフトウェア開発では、**同じアプリを3つの「環境」に置きます**。環境とは「アプリが動く場所のセット（サーバー＋データベース＋設定）」のことです。

> **用語：環境（かんきょう：environment）** アプリが動作する「場所と設定の組み合わせ」。同じプログラムでも、つなぐデータベースや設定値が違えば別の環境になります。

2.1 3つの環境の役割

環境	よみ	場所	何のため	壊しても良い？
ローカル	local	あなたのPC（`localhost`）	開発・実験	◎ いくら壊してもOK
ステージング	staging	本番そっくりのテストサーバー	本番前の最終確認	○ 壊してもユーザーに影響なし
本番	production	Azure の公開サーバー	実際のユーザーが使う	✗ 絶対に壊してはいけない

> **なぜ環境を分けるのか？** いきなり本番でコードを書き換えてテストすると、バグがあったとき **実際のユーザーの業務が止まり、本物のデータが壊れます**。それを防ぐため、「自分のPC（ローカル）で作る → 本番そっくりの場所（ステージング）で確認 → 問題なければ本番へ」という3段階を踏みます。料理でたとえると「自宅で試作 → 試食会で確認 → お店で提供」です。
>
> ⚠️ **本アプリの現状について** 本アプリのリポジトリには、ステージング専用の設定ファイルも、ステージングへ自動で出す仕組みもありません。`frontend/.env.production` にはステージング用の住所（`https://hozen-kosu-another-…azurewebsites.net`）が `#` 付きで書いてあるので、ステージング用のサーバーが用意されていた（いる）ことは分かりますが、GitHub Actions（§14）が自動で出すのは **本番だけ** です。また、`settings.py` の `ALLOWED_HOSTS` には本番と localhost しか無いので、今のコードをそのままステージングに置いても、そのままでは動きません。したがって実態は **ローカル（あなたのPC）** と **本番（Azure）** の 2 環境で、ステージングの代わりに「本番に出す前に、ローカルで本番相当のビルドを動かして確かめる」形で運用します。ステージングのサーバーが今も使えるか・どう出していたかは、リポジトリからは分からないので、前任者に確かめてください（「000 引き継ぎ書」の §10）。この章では、将来ステージングを整える場合の考え方も含めて説明します。

2.2 環境ごとに変わるもの＝設定ファイル

3つの環境で「アプリのコードそのもの」は同じです。違うのは **設定** です。本アプリでは、設定の違いを次のファイルで吸収しています。

ファイル	役割	環境差
`.env`（バックエンド）	DB接続情報・秘密鍵・DEBUG	環境ごとに中身が違う
`frontend/.env.production`	フロントのAPI接続先URL	本番ビルド時のAPI先
`hozen_another/settings.py`	Django全体の設定	`ALLOWED_HOSTS` 等で本番URLを許可

> **用語： `.env` （ドットエンブ：環境変数ファイル）** `KEY=値` の形で秘密情報や環境ごとの設定を書いたファイル。Gitに **入れない**（ `.gitignore` で除外する）のが鉄則。理由は、データベースのパスワードなどがGitHubに公開されると大事故になるからです。`settings.py` の `env('SECRET_KEY')` は、この `.env` から値を読み込んでいます（§11で詳説）。
>
> ⚠️ **`.env` を絶対にGitにコミットしない。**もし誤ってコミットしてしまったら、パスワードや秘密鍵をすぐに変更（ローテーション）してください。Gitの履歴に残った秘密情報は、後から削除しても「一度漏れたもの」として扱う必要があります。

---

3. 日常運用コマンド集

ここでは、後任者が **毎日のように打つコマンド** を、1つずつ意味を添えて紹介します。すべて本アプリの `CLAUDE.md` に記載されている公式コマンドです。

> **用語：コマンド（command：命令）** ターミナル（黒い画面・PowerShell）に打ち込んで、コンピュータに指示を出す1行の文。`Enter` キーで実行されます。

3.1 バックエンド（Django）の起動

> **▼ このコマンドがやること（先に日本語で）**：あなたのPC上でDjangoのWebサーバーを起動し、`http://localhost:8000` でAPIを受け付けられるようにします。

コピー

```
# Djangoの開発用サーバーを localhost:8000 で起動する
python manage.py runserver
```

- `python` … Pythonを実行するコマンド。
- `manage.py` … Djangoプロジェクトを操作する「司令塔」スクリプト。ルート直下にあります。
- `runserver` … 「開発用サーバーを起動せよ」という指示（run＝走らせる、server＝サーバー）。

▼ こう表示されれば成功:

コピー

```
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.
```

この状態でブラウザから `http://localhost:8000` を開くとアプリが表示されます。止めるときは画面で `Ctrl + C` を押します。

⚠️ **`runserver` は開発専用です。**本番では使いません（性能が低く、安全でないため）。本番では Gunicorn（§13）を使います。「開発サーバーを本番で動かす」は禁止です。

3.2 フロントエンド（React）の起動・ビルド

▼ **このコマンドがやること（先に日本語で）**：Reactの開発サーバーを `localhost:3000` で起動します。画面を編集すると即座にブラウザへ反映されます（ホットリロード）。

コピー

```
# Reactの開発サーバーを localhost:3000 で起動（ファイル保存で自動反映）
npm start
# 本番用に最適化したファイル一式を frontend/build/ へ書き出す
npm run build
```

- `npm`（エヌピーエム）… Node.jsのパッケージ管理ツール。Reactのコマンドを実行します。
- `start` … 開発サーバー起動。
- `build` … 本番用ビルド。`build` で作られたファイルを、Djangoがユーザーへ配信します（§11・§13）。

用語：ビルド（build：組み立て） 人間が書いたReact/TypeScriptのソースコードを、ブラウザがそのまま読める形（圧縮されたJavaScript・CSS）に変換すること。`npm run build` を実行すると `frontend/build/` に成果物が出ます。本番デプロイの前に **必ず** 実行します。

3.3 データベースのマイグレーション

用語：マイグレーション（migration：移行・移住） データベースの「表の形（テーブル定義）」を変更する手続き。`models.py` に新しい列を足したり、表を作ったりしたとき、その変更を実際のデータベースへ反映する作業です。引っ越し（移住）で家具の配置図を実際の部屋に当てはめるイメージ。

▼ **このコマンドがやること（先に日本語で）**：`models.py` の変更内容から「設計図（マイグレーションファイル）」を作り、それをデータベースに適用します。

コピー

```
# models.pyの変更から「変更の設計図」を作る
python manage.py makemigrations
# その設計図をデータベースに実際に適用する
python manage.py migrate
```

- `makemigrations` …「変更を検出して設計図ファイルを作れ」。`kosu/migrations/` に番号付きのファイルが生成されます。
- `migrate` …「設計図をデータベースに反映せよ」。実際に表が作られたり列が追加されたりします。

> ⚠️ **順番が大切。**必ず `makemigrations`（設計図を作る）→ `migrate`（適用する）の順です。設計図を作らずに `migrate` しても、何も変わりません。
>
> ⚠️ **本番でのマイグレーションは特に慎重に。**§14で詳しく扱いますが、列を削除するマイグレーションは既存データを失う可能性があります。必ずバックアップを取ってから実行します。

3.4 非同期タスクワーカーの起動

本アプリは「重い処理（バックアップなど）」を裏で動かすために `django-q`（ジャンゴキュー）というしくみを **設定上** 用意しています（§11の `Q_CLUSTER`）。

> ▼ **このコマンドがやること（先に日本語で）：**裏で重い仕事を順番に処理する「ワーカー（働き手）」を起動します。

コピー

```
# django-qのワーカー（裏方の処理係）を起動する
python manage.py qcluster
```

- `qcluster`（キュークラスター）… django-qが提供するコマンド。タスクキュー（仕事の待ち行列）を処理し続ける常駐プロセスを立ち上げます。

用語：ワーカー（worker：働き手） ユーザーのリクエストとは別に、裏で黙々と仕事をこなすプロセス。レストランで「ホール係（リクエスト対応）」と「厨房（ワーカー）」が分かれているイメージ。重い処理を厨房に任せることで、ホール係（Webサーバー）は次のお客さんをすぐ対応できます。

⚠️ **本アプリの実装上の注意（重要）** `settings.py` には `Q_CLUSTER` が定義されており、`django_q` がインストールされていますが、§10で詳しく見るとおり、本アプリのバックアップ処理は実際には **django-qではなく、Pythonの `threading`（スレッド）で別スレッドを起動** して動かしています（`asynchronous_views.py` の `thread = threading.Thread(...)`）。つまり、バックアップ機能自体は `qcluster` を起動していなくても動作します。`Q_CLUSTER` は将来の本格的なタスクキュー化のための土台、あるいは別機能のために残されている設定だと理解してください。この「設定はあるが、実コードはthreading」というギャップは、運用で混乱しやすいポイントなので必ず覚えておきましょう。

3.5 静的ファイルの収集（本番デプロイ時）

▼ このコマンドがやること（先に日本語で）：各所に散らばっている静的ファイル（CSS・JS・画像）を、本番配信用の1か所（`staticfiles/`）に集めます。

コピー

```
# 静的ファイルを STATIC_ROOT（staticfiles/）へ集約する
python manage.py collectstatic
```

- `collectstatic`（コレクトスタティック）… Djangoの管理画面用CSSや、Reactのビルド成果物などを、`settings.py` の `STATIC_ROOT`（§11）が指す `staticfiles/` フォルダへコピーします。WhiteNoise（§11）がここから配信します。

▼ こう表示されれば成功:

コピー

```
XXX static files copied to '.../staticfiles'.
```

⚠️ 本番に出す時の `npm run build`（フロント）→ `collectstatic`（バック）は、`master` に push した時に GitHub Actions が自動で行います（§14.1）。手元で打つのは、手元で本番と同じ形を試したい時だけです。

3.6 コマンド早見表（保存版）

コマンド	何をする	いつ使う
`python manage.py runserver`	Django開発サーバー起動	ローカル開発のたび
`python manage.py migrate`	DBに変更を適用	モデル変更後・初回構築時
`python manage.py makemigrations`	変更の設計図を作る	`models.py` を変えたとき
`python manage.py qcluster`	非同期ワーカー起動	django-qを使う処理が必要なとき
`python manage.py collectstatic`	静的ファイル集約	手元で本番と同じ形を試す時（本番に出す時は GitHub Actions が行う）
`python manage.py createsuperuser`	管理者ユーザー作成	Django管理画面を使う初回
`python manage.py shell`	対話シェル起動	DBの中身を手で調べたいとき
`npm start`	React開発サーバー起動	フロント開発のたび
`npm run build`	フロント本番ビルド	手元で本番と同じ形を試す時（本番に出す時は GitHub Actions が行う）
`npm run test:run`	テストを1回実行	コミット前・確認時
`git push origin master`	GitHub の master に送る	**本番に出す時**（GitHub Actions が自動で本番へ。§14）

---

4. テストの実行

> **用語：テスト（test：試験）** プログラムが「期待どおり動くか」を自動でチェックする小さなプログラム。人間が毎回手で確認する代わりに、機械が一瞬で「OK／NG」を判定してくれます。本アプリのフロントは **Vitest（ビテスト）** というテストツールを使います。

4.1 テストコマンド

> **▼ このコマンドがやること（先に日本語で）：** `frontend/src/tests/` にあるテストを実行し、すべて合格か（緑）・失敗があるか（赤）を表示します。

コピー

```
# 監視モードでテスト（ファイル変更のたびに再実行）
npm test
# テストを1回だけ実行して終了（CI・コミット前向き）
npm run test:run
# ブラウザ上のUIでテスト結果を見る
npm run test:ui
```

▶ ここで初めて出てくるコード

- `npm test`（Node.js）：`package.json` の `"scripts"` の `"test"` を動かします。このアプリでは Vitest が起動し、ファイルを書き換えるたびにテストを流し直します（終わる時は `q`）。

- `npm test` … 「ウォッチモード」。ファイルを保存するたびに自動でテストし直します。開発中に便利。
- `npm run test:run` … 1回実行して結果を出し、終了します。「今のコードが壊れていないか」を確認する用途。
- `npm run test:ui` … テスト結果をブラウザの見やすい画面で確認できます。

▼ こう表示されれば成功:

コピー

```
✓ src/tests/Login.test.tsx (3)
Test Files  1 passed (1)
     Tests  3 passed (3)
```

`passed`（合格）が並べばOK。`failed`（失敗）が出たら、その下にどのテストがなぜ落ちたかが表示されます。

4.2 テストはいつ実行するか

タイミング	実行するコマンド	理由
開発中ずっと	`npm test`（ウォッチ）	壊した瞬間に気づける
コミット前	`npm run test:run`	壊れたコードを保存しないため
デプロイ前	`npm run test:run`	本番に壊れたコードを出さないため

なぜ？ テストは「過去の自分・他人が作った機能を、今の変更で壊していないか」を保証する **安全ネット** です。後任者にとって特に重要で、「触ったことのない場所を変更したらテストが赤くなった」という形で、思わぬ影響を教えてくれます。

5. バックアップ／復元機能の全体像

ここからが本章の核心です。本アプリには「データをExcelに書き出す（バックアップ）」「Excelからデータを戻す（ロード／復元）」「期間指定でデータを消す（削除）」機能があります。これらが **運用で最もよく使う機能** です。

5.1 全体の流れ（絵で理解する）

> **▼ この図がやること（先に日本語で）**：ユーザーがボタンを押してから、Excelをダウンロードするまでの一連の流れを示します。



図9-2: 工数のバックアップ（ダウンロード）の流れ。受付番号（task_id）をすぐ返し、重い仕事は別のスレッドで進める

> **用語：非同期（ひどうき：asynchronous）**「頼んだ仕事の完了を待たずに、先に進む」やり方。バックアップは重い（数秒〜数分かかる）ので、ユーザーを待たせ続けると画面が固まります。そこで「受付番号（タスクID）だけ先に返して、処理は裏で進め、完了したら番号で結果を取りに来てもらう」方式にしています。これが非同期です。レストランの「呼び出しベル」と同じ。番号札を受け取って席で待ち、できたら番号で呼ばれる。
>
> **用語：ポーリング（polling：定期確認）**「終わった？ まだ？」と一定間隔で何度も問い合わせること。本アプリのフロントは `check_task_status` を数秒おきに呼び、`status` が `pending`（処理中）から `success` ／ `error` に変わるのを待ちます。

5.2 関係する3つのファイルと役割

ファイル	役割	この章での節
`kosu/tasks.py`	実際にExcelを作る・DBを読み書きする「作業の中身」	§6〜§9
`kosu/views/asynchronous_views.py`	ボタンの受付・スレッド起動・状態確認・配信	§10
`kosu/models.py` の `AsyncTask`	タスクの状態（受付番号・成否・結果）を保存する表	§10で参照

それでは、作業の中身である `tasks.py` から1行ずつ読んでいきます。

---

6. tasks.py 全解説①：工数バックアップ（openpyxl）

6.1 ファイル冒頭のインポート

> **▼ このコードがやること（先に日本語で）**：このファイルで使う道具（部品）を、あちこちから持ってきます。Excel操作・表計算・タイムゾーン・日付・データベースのモデルなどです。

コピー

```
# ファイル・フォルダ操作（パス結合・存在確認・削除）
import os
# settings.py の値（MEDIA_ROOT等）を読むため
from django.conf import settings
# タイムゾーン付きの時刻を扱う道具
from django.utils import timezone
# Excel（.xlsx）を読み書きするライブラリ
import openpyxl
# 表形式データを扱うライブラリ（pdという短い別名で使う）
import pandas as pd
# タイムゾーン変換（Asia/Tokyo等）の道具
import pytz
# 日付・時刻を扱う標準ライブラリ
import datetime
from .models import Business_Time_graph, kosu_division, def_choice, member, team_member, \
                    # 使うDBモデル一式
                    inquiry_data, administrator_data, AsyncTask, History
# 工数区分の変換辞書を作る自作関数
from .utils.kosu_utils import kosu_division_dictionary
# DB整合性エラー（重複登録等）を捕まえる例外クラス
from django.db import IntegrityError
# 「該当データなし」例外クラス
from django.core.exceptions import ObjectDoesNotExist
```

▶ ここで初めて出てくるコード

- **`IntegrityError`**（Django）：データベースの決まり（同じ値は 2 つ入れない、など）を破った時のエラー。`except IntegrityError:` で受け止めます。
- **`ObjectDoesNotExist`**（Django）：`get()` で探した行が見つからない時のエラー。`except ObjectDoesNotExist:` で受け止めます。

1行ずつ意味を補足します。

- `import os` … `os` は Operating System（オペレーティングシステム）の略。ファイルやフォルダを操作します。後で `os.path.join`（パスをつなぐ）・`os.path.exists`（存在確認）・`os.makedirs`（フォルダ作成）・`os.remove`（削除）で大活躍します。
- `from django.conf import settings` … `settings.py`（§11）の設定値を読み込むための入口。`settings.MEDIA_ROOT`（保存先フォルダ）・`settings.TIME_ZONE`（タイムゾーン）を後で使います。
- `from django.utils import timezone` … タイムゾーンを意識した「今」を取る道具。本ファイルでは直接の出番は少ないですが、Djangoの時刻処理の標準です。
- `import openpyxl` … **本章の主役の1つ。**Excelファイル（拡張子 `.xlsx`）をPythonから作ったり読んだりできるライブラリ。「ブック（ファイル）→シート→セル」という構造を扱います。

- `import pandas as pd` … **もう1つの主役**。表（行と列）のデータを高速に処理するライブラリ。`as pd` は「これ以降 `pandas` を `pd` と短く書く」という別名指定。`generate_prediction`（§7）で重複削除に使います。
- `import pytz` … Python TimeZone（パイティーズィー）。世界中のタイムゾーン情報を持つライブラリ。UTC（協定世界時）で保存された時刻を日本時間に直すのに使います。
- `import datetime` … 日付（`date`）や日時（`datetime`）を扱う標準ライブラリ。`datetime.date.today()`（今日）などで使用。
- `from .models import ...` … 同じ `kosu` アプリの `models.py` から、DBの表に対応する「モデルクラス」を一括で取り込みます。`\`（バックスラッシュ）は「行が続く」という意味の継続記号。

> **用語：モデル（model：模型・型）** Djangoで「データベースの1つの表」を表すPythonのクラス。例えば `member` クラスは「人員テーブル」に対応し、1人の人員が1行（1インスタンス）です。`member.objects.all()` と書くと「人員テーブルの全行」を取り出せます。

- `from .utils.kosu_utils import kosu_division_dictionary` … 工数区分の記号（例：`A`）を、読みやすい名前へ変換する辞書を作る自作関数。§7で使います。
- `from django.db import IntegrityError` … データベースの「整合性エラー」（例：同じ主キーを2回登録しようとした）を捕まえるための例外クラス。ロード処理でエラーをスキップするのに使います。
- `from django.core.exceptions import ObjectDoesNotExist` … 「探したデータが存在しない」ときに投げられる例外。ロード処理で「該当する人員がいない行はスキップ」に使います。

> **用語：例外（れいがい：exception）** プログラム実行中に起きる「想定外の事態」を表す合図。例えば「ファイルがない」「DBに重複がある」など。`try:`（試す）と `except:`（捕まえる）で、エラーが起きてもプログラム全体を止めずに対処できます。

6.2 工数データバックアップ関数 `generate_kosu_backup`

これが「工数データをExcelに書き出す」関数です。バックアップ機能の基本形なので、特に丁寧に読みます。

> **▼ このコードがやること（先に日本語で）**：指定された開始日〜終了日の工数データをDBから全部取り出し、新しいExcelファイルを作って、見出し行とデータ行を書き込み、`media/` フォルダに保存して、そのファイルの場所（パス）を返します。

コピー

```
# 工数データバックアップ非同期処理
# 開始日と終了日を受け取る関数
def generate_kosu_backup(start_day, end_day):
  # 新しいExcelブック作成
  # 空っぽのExcelブック（ファイル）を新規作成
  wb = openpyxl.Workbook()
  # その中の「最初のシート」を取り出す
  ws = wb.active
```

▶ ここで初めて出てくるコード

- `.Workbook()`（pandas・openpyxl）：`openpyxl.Workbook()` は、新しい空の Excel のファイル（ブック）を作ります。`wb.active` が最初のシートです。

- `def generate_kosu_backup(start_day, end_day):` … `def`（define＝定義）で関数を作ります。`start_day`（開始日）・`end_day`（終了日）の2つの引数を受け取ります。これらは `asynchronous_views.py`（§10）から渡されてきます。
- `wb = openpyxl.Workbook()` … 新しいExcelブック（ファイル全体）をメモリ上に作ります。`wb` は workbook（ワークブック＝ブック）の略。この時点ではまだディスクには保存されていません。
- `ws = wb.active` … ブックの中の「アクティブなシート（＝最初のシート）」を取り出して `ws` に入れます。`ws` は worksheet（ワークシート）の略。以降、このシートに行を書き込んでいきます。

用語：openpyxlの3階層 Excelは「ブック（ファイル）→ ワークシート（タブ）→ セル（マス目）」の3階層です。`Workbook()` でブックを作り、`wb.active` でシートを取り、`ws.append([...])` で行を追加し、`ws.cell(行, 列)` でマス目1つを読みます。この3つを覚えれば openpyxl の8割は使えます。

次に、Excelの1行目（見出し）を用意します。

コピー

```
# ヘッダー作成
headers = [
  # 1〜5列目の見出し
  '従業員番号', '氏名', '工数区分定義Ver', '就業日', '直',
  # 6〜9列目
  '作業内容', '作業詳細', '残業時間', '昼休憩時間',
  # 10〜12列目
  '残業休憩時間1', '残業休憩時間2', '残業休憩時間3',
  # 13〜15列目
  '就業形態', '工数入力OK_NG', '休憩変更チェック',
  ]
# この見出しリストをExcelの1行目として追加
ws.append(headers)
```

- `headers = [...]` … リスト（角カッコ `[ ]` で囲んだ並び）で、Excelの1行目に書く列見出しを定義します。全部で15個あります。
- `ws.append(headers)` … `append`（アペンド＝末尾に追加）で、このリストを **シートの次の空き行（最初なので1行目）** にまるごと書き込みます。1要素が1セルになります。

> ⚠️ **見出しの順番と数は「ロード」と完全に一致させる必要があります。** バックアップで書き出した順番と、ロード（§8）で読み込む順番がズレると、データが別の列に入ってしまいます。`load_kosu_file` の `expected_headers`（§8）と見比べると、まったく同じ並びになっています。これは偶然ではなく、**バックアップで出したものをそのままロードで戻せるようにする** ための設計です。

次に、データベースから対象データを取り出します。

コピー

```
# 工数データ取得
# 工数テーブルから条件で絞り込み
kosu_data = Business_Time_graph.objects.filter(
  # 就業日が start_day 以上 かつ end_day 以下
  work_day2__gte=start_day, work_day2__lte=end_day
  )
```

- `Business_Time_graph.objects.filter(...)` … 工数の表（`Business_Time_graph` モデル）から、条件に合う行だけを取り出します。`objects` は「この表の全レコードを操作する窓口」、`.filter(...)` は「条件で絞る」命令です。
- `work_day2__gte=start_day` … `work_day2`（就業日の列）が `start_day` **以上** という条件。`__gte` は greater than or equal（〜以上）の略。
- `work_day2__lte=end_day` … `work_day2` が `end_day` **以下** という条件。`__lte` は less than or equal（〜以下）の略。

- カンマで並べた2条件は **両方とも満たす（AND）** という意味になります。つまり「`start_day ≦ 就業日 ≦ end_day`」の範囲です。

> **用語：QuerySet（クエリセット）** `.filter()` が返す「絞り込み結果の集まり」。この時点ではまだDBに問い合わせていません（遅延評価）。`for` で回したり `.exists()` で確認したりして **実際に値が必要になった瞬間** にDBへ問い合わせます。料理の「注文票」のようなもので、注文を書いた時点ではまだ料理は作られず、必要になって初めて厨房が動きます。

取り出したデータを1行ずつExcelへ書き込みます。

コピー

```
# データ書き込み
# 取得した工数データを1件ずつ取り出す
for item in kosu_data:
  # Excel1行ぶんのデータをリストにまとめる
  row = [
    # 従業員番号・氏名・Ver・就業日
    item.employee_no3, str(item.name), item.def_ver2, item.work_day2,
    # 直・作業内容・作業詳細・残業時間
    item.tyoku2, item.time_work, item.detail_work, item.over_time,
    # 昼休憩・残業休憩1・残業休憩2
    item.breaktime, item.breaktime_over1, item.breaktime_over2,
    # 残業休憩3・就業形態・入力OK_NG
    item.breaktime_over3, item.work_time, item.judgement,
    # 休憩変更チェック
    item.break_change,
    ]
  # この1行をExcelシートの末尾に追加
  ws.append(row)
```

- `for item in kosu_data:` … `kosu_data`（絞り込んだ工数データ）を **1件ずつ** `item` に入れて繰り返します。`item` は1人の1日ぶんの工数レコードです。
- `row = [...]` … `item` の各フィールド（列の値）を取り出して、Excel1行ぶんのリストにまとめます。`item.employee_no3` は従業員番号、`item.work_day2` は就業日、というように **モデルのフィールド名** で値を取ります。
- `str(item.name)` … `item.name` は人員モデルへの参照（オブジェクト）なので、`str(...)` で **文字列（氏名）** に変換しています。これをしないと「member object (5)」のような表示になってしまいます。
- `ws.append(row)` … この1行をシートの末尾に追加。`for` で全件繰り返すので、データの数だけ行が増えていきます。

> **なぜ `name` だけ `str()` するのか？** `employee_no3` などは数値や文字列ですが、`name` は「人員テーブルへのリンク（外部キー）」です。Pythonのオブジェクトのままでは人間が読めないので、`str()` を通して `__str__` メソッド（モデルで定義された「氏名を返す」表示用関数）を呼び、氏名の文字列にしているのです。

書き込みが終わったら、保存先フォルダを準備して保存します。

コピー

```
# 保存先のディレクトリ確認・作成
# settings.pyのMEDIA_ROOT（media/フォルダのパス）を取得
media_dir = settings.MEDIA_ROOT
# もしそのフォルダが存在しなければ
if not os.path.exists(media_dir):
  # フォルダを作る
  os.makedirs(media_dir)
```

> **▶ ここで初めて出てくるコード**
>
> - `.makedirs()`（Python）：`os.makedirs(フォルダ)` は、フォルダを作ります。途中のフォルダが無ければ、それも作ります。

- `media_dir = settings.MEDIA_ROOT` … 保存先フォルダのパスを `settings.py` から取得します（`MEDIA_ROOT` は §11で `BASE_DIR/media` と定義）。
- `if not os.path.exists(media_dir):` … `os.path.exists` は「そのパスが存在するか」を `True/False` で返します。先頭の `not` で「存在しないなら」という条件になります。
- `os.makedirs(media_dir)` … フォルダを新規作成します。`makedirs` は途中のフォルダもまとめて作れる関数。**初回バックアップ時に `media/` フォルダがまだ無くてもエラーにならない** ための保険です。

コピー

```
# ファイル名作成と保存
# ファイル名を組み立て
filename = f'工数データバックアップ_{start_day}_{end_day}.xlsx'
# フォルダパスとファイル名を結合して完全なパスに
filepath = os.path.join(media_dir, filename)
# Excelブックをそのパスに保存（ここで初めてディスクに書かれる）
wb.save(filepath)

# ファイルパスを返却
# 保存したファイルの場所を呼び出し元へ返す
return filepath
```

- `filename = f'...{start_day}_{end_day}.xlsx'` … `f'...'` は **f-string（エフストリング）**。`{ }` の中に変数の値を埋め込めます。たとえば `start_day=2026-05-01`、`end_day=2026-05-31` なら、ファイル名は `工数データバックアップ_2026-05-01_2026-05-31.xlsx` になります。
- `filepath = os.path.join(media_dir, filename)` … フォルダのパスとファイル名を、OSに合った区切り文字（Windowsなら `\`、Linuxなら `/`）で正しくつなぎます。文字列を `+` でつなぐより安全です。
- `wb.save(filepath)` … メモリ上で組み立てたExcelブックを、実際にディスクへ保存します。**この行で初めてファイルが生まれます。**
- `return filepath` … 保存したファイルの場所を、呼び出し元（§10の `handle_task`）へ返します。この戻り値が `AsyncTask.result` に保存され、後でダウンロードに使われます。

> **▼ こう動けば成功:** この関数が終わると、`media/工数データバックアップ_2026-05-01_2026-05-31.xlsx` というExcelファイルができ、その絶対パスが返ります。ユーザーはこのあと §10 のダウンロード処理でこのファイルを受け取ります。
>
> **この関数のパターンを覚えると、他もすぐ読める。** `tasks.py` の `generate_member_backup`・`generate_team_backup`・`generate_def_backup` などはすべて **同じ骨組み**（ブック作成→ヘッダー→データ取得→行追加→フォルダ確認→保存→パス返却）です。違うのは「どの表から、どの列を取るか」だけ。だから §6 を理解すれば §9 はほぼ読めます。

---

7. tasks.py 全解説②：予測データ出力（pandas）

次は少し毛色の違う `generate_prediction` です。これは「工数データから作業詳細を抜き出し、重複を消して一覧化する」処理で、**pandas** を使うのが特徴です。

> **▼ このコードがやること（先に日本語で）:** 指定期間の工数データを取り出し、各レコードの「作業内容（記号の並び）」と「作業詳細（$区切りの文字）」を1つずつ対応させて、読みやすい区分名に変換しながらリスト化し、pandasの表（DataFrame）にして、空欄や重複を取り除いてExcelに出力します。

7.1 データ取得と準備

コピー

```
# 工数区分定義予測データ出力非同期処理
# 開始日・終了日を受け取る
def generate_prediction(data_day, data_day2):
  # 期間内の工数データ取得
  # 工数テーブルを期間で絞る
  kosu_filter = Business_Time_graph.objects.filter(
    # data_day 以上 data_day2 以下
    work_day2__gte=data_day, work_day2__lte=data_day2
    )

  # Excelに書き出すためのデータを準備
  # 結果を貯める空のリストを用意
  data = []
```

- `def generate_prediction(data_day, data_day2):` … 開始日 `data_day` と終了日 `data_day2` を受け取ります。
- `kosu_filter = Business_Time_graph.objects.filter(...)` … §6と同じく、期間で工数データを絞り込みます。
- `data = []` … 空のリスト。ここに「区分名・作業詳細」のペアを溜めていきます。

7.2 1レコードを記号ごとに分解する

コピー

```
  # 取得した工数データを処理
  # 工数レコードを1件ずつ
  for kosu in kosu_filter:
    # 作業内容をリストに解凍
    # 作業内容（記号の文字列）を1文字ずつのリストに
    kosu_list = list(kosu.time_work)
    # 作業詳細をリストに解凍
    # 作業詳細を「$」で区切ってリストに
    detail_list = kosu.detail_work.split('$')
    # 工数定義区分取得
    # この工数のVerに対応する定義を取得
    def_filter = kosu_division.objects.filter(kosu_name=kosu.def_ver2)
```

▶ ここで初めて出てくるコード

- `.split()` （Python）: Python で、文字列を分けてリストにするメソッド。`"a b c".split()` のように何も渡さないと、空白（いくつ続いても）で分けます。

- `for kosu in kosu_filter:` … 工数レコードを1件ずつ `kosu` に取り出します。
- `kosu_list = list(kosu.time_work)` … `kosu.time_work`（作業内容）は「`AABBC`」のような **記号が連なった1つの文字列**。`list(...)` でこれを1文字ずつのリスト `['A','A','B','B','C']` に分解します。各文字が「その時間に何をしたか」の記号です。
- `detail_list = kosu.detail_work.split('$')` … `kosu.detail_work`（作業詳細）は「`部品交換$点検$$清掃`」のように `$` **で区切られた文字列**。`.split('$')` で区切り、`['部品交換','点検','','清掃']` のようなリストにします。
- `def_filter = kosu_division.objects.filter(kosu_name=kosu.def_ver2)` … この工数が使っている「工数区分定義のバージョン名（`def_ver2`）」と一致する定義を、定義テーブルから取得します。記号 `A` が実際には「何の作業か」を引くための辞書のもとになります。

> **用語：split（スプリット：分割）** 文字列を、指定した区切り文字（ここでは `$`）でちぎってリストにするメソッド。逆に、リストを区切り文字でつなぐのは `'$'.join(リスト)` です。本アプリは「複数の値を1つの列に詰める」ために `$` 区切りを多用しています。

7.3 記号を読みやすい名前に変換する

コピー

```
    # リストの長さ取得
    # 2つのリストのうち長い方の長さ
    max_length = max(len(kosu_list), len(detail_list))
    # 1要素ごとにExcelに書き込み
    # 0 から max_length-1 まで繰り返す
    for i in range(max_length):
      # 工数定義区分がある場合の処理
      # 対応する定義が存在するなら
      if def_filter.exists():
        # 工数定義区分リスト作成
        # 記号→名前の対応リストを作る
        choices_list, def_n = kosu_division_dictionary(kosu.def_ver2)
```

> **▶ ここで初めて出てくるコード**
>
> - **`max()`**（Python）：いちばん大きい物を返す関数。`max(3, 9, 4)` は 9、`max([5, 1, 8])` は 8。いちばん小さい物は `min()` です。

- `max_length = max(len(kosu_list), len(detail_list))` … 作業内容と作業詳細、**長い方の要素数**を取ります。`len(...)` は要素数、`max(a, b)` は大きい方を返します。両者の長さがズレていても全要素を見るための工夫です。
- `for i in range(max_length):` … `0, 1, 2, ... max_length-1` と番号を振りながら繰り返します。`i` が「何番目の作業か」のインデックスです。

- `if def_filter.exists():` … `.exists()` は「絞り込み結果が1件でもあるか」を `True/False` で返す軽い確認。定義が存在するときだけ中の処理をします。
- `choices_list, def_n = kosu_division_dictionary(kosu.def_ver2)` … 自作関数で「記号 → 読みやすい名前」の対応表（`choices_list`）を作ります。`def_n` は使う定義数。`choices_list` の各要素は `(記号, 名前)` のペアになっています。

コピー

```
        # 作業内容を工数区分定義に変換
        # 対応表を1ペアずつ
        for k in choices_list:
          # 工数区分定義の記号と作業内容が同じ場合の処理
          # ペアの記号が、i番目の作業内容記号と一致したら
          if k[0] == kosu_list[i]:
            # 作業詳細が空欄でない場合の処理
            # i番目の作業詳細が空でないなら
            if detail_list[i] != '':
              # 作業内容と作業詳細をセットで定義
              row = [
                  # 区分名（範囲内なら名前、外なら空）
                  k[1] if i < len(kosu_list) else '',
                  # 作業詳細（同上）
                  detail_list[i] if i < len(detail_list) else '',
              ]
              # 作業内容と作業詳細を書き込み
              # 結果リストに1ペア追加
              data.append(row)
              #ループから抜ける
              # 一致したら内側ループを抜ける（無駄な探索をやめる）
              break
```

- `for k in choices_list:` … 対応表のペア `k`（`(記号, 名前)`）を1つずつ調べます。
- `if k[0] == kosu_list[i]:` … `k[0]`（ペアの記号）と、`kosu_list[i]`（i番目の作業内容記号）が一致するかを確認。一致すれば「この記号の正体（名前）が分かった」ということです。
- `if detail_list[i] != '':` … i番目の作業詳細が空文字でなければ処理。空の作業詳細は記録する意味がないので除外します。
- `row = [k[1] if ... else '', detail_list[i] if ... else '']` … `k[1]`（区分名）と `detail_list[i]`（作業詳細）のペアを作ります。`A if 条件 else B` は **三項演算子**で「条件が真ならA、偽ならB」。ここでは `i` がリストの範囲内かを確認し、範囲外なら空文字にする安全策です。
- `data.append(row)` … このペアを結果リスト `data` に追加します。
- `break` … 一致するペアが見つかったら、それ以上 `choices_list` を探す必要はないので内側の `for` を抜けます。効率化のためです。

> **用語：三項演算子（さんこうえんざんし）** `A if 条件 else B` 「条件が真ならA、偽ならB」を1行で書く構文。`if/else` を短く書いたもの。`x = '大人' if age >= 18 else '子供'` のように使います。（※説明用の簡易例）

7.4 pandasで表にして重複・空欄を消す

ここで pandas の出番です。

コピー

```
# DataFrameに変換
# リストを表(DataFrame)に変換
df = pd.DataFrame(data, columns=['工数定義区分', '作業詳細'])

# フィルタリング条件に基づいて不要な行を削除
# 区分が空・#・$ の行を除外
df = df[~df['工数定義区分'].isin(['', '#', '$'])]
# 作業詳細が空の行を除外
df = df[df['作業詳細'] != '']

# 工数定義区分と作業詳細の重複行を削除
# 同じ組み合わせは最初の1つだけ残す
df = df.drop_duplicates(subset=['工数定義区分', '作業詳細'], keep='first')
```

> **▶ ここで初めて出てくるコード**
>
> - `.DataFrame()`（pandas・openpyxl）：`pd.DataFrame(データ, columns=[列の名前…])` は、pandas の表（データフレーム）を作ります。Excel のシートのように、行と列で値を持ちます。
> - `.isin()`（pandas・openpyxl）：`df['列'].isin(['', '#'])` は、その列の値が並びの中のどれかなら `True` を返します。前に `~` を付けると「どれでもない」になります。
> - `.drop_duplicates()`（pandas・openpyxl）：重なっている行を消します。`subset=[列…]` で「この列が同じなら重なり」とし、`keep='first'` で最初の 1 行を残します。

- `df = pd.DataFrame(data, columns=[...])` … `data`（ペアのリスト）を、pandasの **DataFrame（データフレーム＝表）** に変換します。`columns` で列名を指定。これでExcelのような「行×列」のデータになります。
- `df = df[~df['工数定義区分'].isin(['', '#', '$'])]` … `isin([...])` は「その列の値が、リストのどれかに一致するか」を行ごとに判定。先頭の `~`（チルダ）は **否定（NOT）**。つまり「区分が `''`・`#`・`$` の **いずれでもない** 行だけを残す」という意味です。`#` や `$` は区切り記号のゴミなので除外します。
- `df = df[df['作業詳細'] != '']` … 作業詳細が空文字でない行だけを残します。
- `df.drop_duplicates(subset=[...], keep='first')` … `drop_duplicates`（重複削除）で、「工数定義区分」と「作業詳細」の組み合わせが同じ行を、**最初の1つだけ残して** 消します。`keep='first'` が「最初を残す」指定。これで「同じ作業詳細が何百回も出る」のを1行にまとめ、すっきりした一覧になります。

用語：DataFrame（データフレーム） pandasの中心となる「表」のデータ構造。Excelのシートのように「行（レコード）」と「列（フィールド）」を持ち、`df[条件]` で行を絞ったり、`df.drop_duplicates()` で重複を消したりが一瞬でできます。手で `for` を回すより速く・短く書けるのが利点です。

7.5 保存（pandas経由）

コピー

```
# 保存先のディレクトリ確認・作成
# 保存先フォルダ
media_dir = settings.MEDIA_ROOT
# 無ければ
if not os.path.exists(media_dir):
  # 作る
  os.makedirs(media_dir)

# ファイル名作成と保存
# ファイル名
filename = f'工数予測データバックアップ_{data_day}_{data_day2}.xlsx'
# 完全なパス
filepath = os.path.join(media_dir, filename)

# Excelファイルをメモリ上で作成して保存
# ExcelWriterを開く
with pd.ExcelWriter(filepath, engine='openpyxl') as writer:
    # DataFrameをExcelに書き出す（行番号は出さない）
    df.to_excel(writer, index=False)

# ファイルパスを返却
# 保存パスを返す
return filepath
```

▶ ここで初めて出てくるコード

- **`.ExcelWriter()`**（pandas・openpyxl）：`pd.ExcelWriter(ファイルの場所, engine='openpyxl')` は、pandas の表を Excel のファイルに書く係を作ります。`with` で使うと、最後に自動で保存して閉じます。
- **`.to_excel()`**（pandas・openpyxl）：`df.to_excel(writer, index=False)` は、表を Excel に書きます。`index=False` で、左の行番号の列を書きません。

- フォルダ確認・ファイル名組み立ては §6 と同じパターンです。
- `with pd.ExcelWriter(filepath, engine='openpyxl') as writer:` … pandasがExcelに書き出すための「書き込み係」を開きます。`engine='openpyxl'` で内部的にopenpyxlを使う指定。`with ... as ...:` を使うと、**ブロックを抜けるときに自動でファイルを閉じて保存** してくれます（閉じ忘れ防止）。

- `df.to_excel(writer, index=False)` … DataFrameをExcelファイルに書き出します。`index=False` は「pandasが内部で持っている行番号（インデックス）は出力しない」指定。これがないと、左端に余計な番号列が付いてしまいます。

> **なぜ §6 は `wb.save`、§7 は `ExcelWriter` なのか？** §6 は openpyxl で1行ずつ手作業で組み立てたので、最後に `wb.save()` で保存します。§7 は pandas の DataFrame として持っているので、pandas に「これをExcelにして」と頼む `to_excel` を使います。**どちらも最終的にopenpyxlがExcelファイルを書いている** 点は同じです。

---

8. tasks.py 全解説③：ロード（Excel→DB）と削除

バックアップ（DB→Excel）の逆向き、**ロード（Excel→DB）** を読みます。これは「壊れたデータを過去のバックアップから戻す」復元作業の中身です。

8.1 工数データ削除 `delete_kosu_data`

まず最も短い「削除」から。復元の前に古いデータを消す、あるいは不要な期間を整理するのに使います。

> **▼ このコードがやること（先に日本語で）**：指定期間の工数データをDBから取り出して、まとめて削除します。

コピー

```
# 工数データ削除非同期処理
# 開始日・終了日を受け取る
def delete_kosu_data(start_day, end_day):
  # 工数データ取得
  # 期間で絞る
  kosu_obj = Business_Time_graph.objects.filter(work_day2__gte=start_day, work_day2__lte=end_day)
  # 取得した工数データを削除
  # 絞った行をまとめて削除
  kosu_obj.delete()

  # 戻り値なし（成功扱い）
  return None
```

- `kosu_obj = ...filter(...)` … §6と同じく期間で絞ります。
- `kosu_obj.delete()` … 絞り込んだ行を **一括削除** します。`.delete()` は QuerySet 全体を1回のSQLで消すので高速です。
- `return None` … 戻り値はありません。`handle_task`（§10）はこれを「正常終了」として扱います。

> ⚠️ **`.delete()` は取り消せません。** 一度消したデータは戻せません。本アプリでは、削除画面でも `validate_dates`（§10）により「昨日までの日付しか指定できない」など歯止めがありますが、それでも削除前にバックアップを取るのが鉄則です。

8.2 工数データロード `load_kosu_file`

Excelを読んでDBへ流し込む処理です。やや長いですが、エラー処理を含めて1つずつ読みます。

> **▼ このコードがやること（先に日本語で）**：受け取ったExcelファイルを開き、見出しが正しいか確認し、2行目以降のデータを1行ずつ読み、対応する人員が存在すれば、同じ日の既存データを消してから新しいデータを登録します。最後に一時ファイルを削除します。

コピー

```
# 工数データロード非同期処理
# アップロードされた一時ファイルのパスを受け取る
def load_kosu_file(file_path):
  # 以下の処理でエラーが出ても落ちないように包む
  try:
    # 一時ファイルパスを変数に保持
    temp_file_path = file_path
    # Excelファイルを開く
    wb = openpyxl.load_workbook(temp_file_path)
    # 最初のシートを取得
    ws = wb.worksheets[0]
```

> **▶ ここで初めて出てくるコード**
>
> - `.load_workbook()`（pandas・openpyxl）：`openpyxl.load_workbook(ファイルの場所)` は、Excel のファイルを読み込みます。

- `def load_kosu_file(file_path):` … 引数は「アップロードされたExcelの一時保存パス」。`asynchronous_views.py`（§10）が一時ファイルを作って渡してきます。
- `try:` … この先の処理で例外が起きても、プログラム全体を止めず `except`（後述）で受け止めるための囲い。
- `wb = openpyxl.load_workbook(temp_file_path)` … `load_workbook` は「既存のExcelファイルを読み込む」関数（作成の `Workbook()` と対）。
- `ws = wb.worksheets[0]` … `worksheets[0]` は「0番目（最初）のシート」。`wb.active` と同様に最初のシートを取ります。

コピー

```
    # ヘッダー定義
    expected_headers = [
      '従業員番号', '氏名', '工数区分定義Ver', '就業日', '直',
      '作業内容', '作業詳細', '残業時間', '昼休憩時間',
      '残業休憩時間1', '残業休憩時間2', '残業休憩時間3',
      '就業形態', '工数入力OK_NG', '休憩変更チェック',
    ]

    # 実ファイルの1行目を取得
    actual_headers = [ws.cell(1, col).value for col in range(1, len(expected_headers) + 1)]

    # ヘッダーが一致しない場合、一時ファイルを削除しエラーを返す
    # 期待する見出しと違ったら
    if actual_headers != expected_headers:
      # 一時ファイルを消して
      os.remove(temp_file_path)
      # エラーを返す
      return {'status': 'error', 'message': '無効なファイルフォーマットです。'}, None
```

▶ ここで初めて出てくるコード

- `.cell()`（pandas・openpyxl）：`ws.cell(行, 列)` は、シートの 1 つのマスを取り出します（どちらも 1 から数えます）。`.value` がマスの中身です。

- `expected_headers = [...]` … **期待する見出し**。§6のバックアップで書き出した見出しと **完全に同じ並び** です。これと違うファイルは「フォーマットが違う」と判断します。
- `actual_headers = [ws.cell(1, col).value for col in range(1, len(expected_headers) + 1)]` … **リスト内包表記**。`range(1, 16)` で列番号1〜15を回し、`ws.cell(1, col).value`（1行目・col列目のセルの値）を集めて、実ファイルの見出しリストを作ります。
- `if actual_headers != expected_headers:` … 実際の見出しと期待する見出しが **食い違ったら**、エラーにします。
- `os.remove(temp_file_path)` … 不正なファイルなのでサーバーから消します。
- `return {'status': 'error', ...}, None` … **タプル（2要素の組）** を返します。1つ目が辞書 `{'status': 'error', ...}`、2つ目が `None`。`handle_task`（§10）はこの「1つ目が `status='error'` の辞書」を見て、エラー終了と判断します。

用語：リスト内包表記（list comprehension） `[式 for 変数 in 範囲]` の形で、リストを1行で作る書き方。`[ws.cell(1, col).value for col in range(1, 16)]` は「1〜15の各 col について、1行目col列のセル値を集めたリスト」です。`for` ループで `append` するのと同じことを短く書けます。

用語： `ws.cell(行, 列)` openpyxlでセル1つを指す書き方。**行も列も1から始まる**（Excelと同じ）ことに注意。`ws.cell(1, 1)` はA1セル、`ws.cell(2, 3)` はC2セルです。`.value` でその中身を取り出します。

データの読み込みとDB保存に進みます。

コピー

```
# 4. データ読み込みとDB保存
# 2行目から最終行まで（1行目は見出しなので飛ばす）
for i in range(2, ws.max_row + 1):
  # 1列目＝従業員番号
  employee_no = ws.cell(row=i, column=1).value
  # 4列目＝就業日
  work_day2 = ws.cell(row=i, column=4).value

  try:
    # その従業員番号の人員を取得
    member_instance = member.objects.get(employee_no=employee_no)
  # 見つからなかったら
  except ObjectDoesNotExist:
    print(f"警告: 従業員番号 {employee_no} (行 {i}) のメンバーが見つかりませんでした。スキップします。")
    # 次のループへ進む            # この行は飛ばして次へ
    continue
```

- `for i in range(2, ws.max_row + 1):` … `i` を **2** から `ws.max_row`（最終行）まで回します。1行目は見出しなので **2行目から** データとして読むのです。`+1` は range が「終端を含まない」ためで、最終行まで確実に回すための調整です。
- `employee_no = ws.cell(row=i, column=1).value` … i行目・1列目（従業員番号）を取得。
- `work_day2 = ws.cell(row=i, column=4).value` … i行目・4列目（就業日）を取得。
- `member_instance = member.objects.get(employee_no=employee_no)` … `.get(...)` は「条件に **ぴったり1件** 該当する行を取得」。その従業員番号の人員を1人取ります。
- `except ObjectDoesNotExist:` … もしその従業員番号の人員が **存在しなければ** ここに来ます。`.get()` は該当なしだと例外を投げるからです。
- `print(...)` … 警告メッセージをログに出します（`f"..."` で行番号や番号を埋め込み）。
- `continue` … `continue` は「この回の繰り返しを中断して、**次の行へ進む**」命令。人員がいない行は登録できないので飛ばします。

> **用語： `.get()` と `.filter()` の違い** `.filter()` は条件に合う行の **集まり（0件でもOK）** を返します。`.get()` は **ちょうど1件** を返し、0件だと `DoesNotExist` 例外、2件以上だと `MultipleObjectsReturned` 例外を投げます。「1件だけ確実に欲しい」ときは `.get()`、「複数あるかも／0件かも」なら `.filter()` を使います。

コピー

```
        # もし既に同一データが存在するなら削除
        existing_data = Business_Time_graph.objects.filter(
          # 同じ人・同じ日の既存データ
          employee_no3=employee_no, work_day2=work_day2
          )
        # 既存があれば
        if existing_data.exists():
          # 消す（重複防止）
          existing_data.delete()
```

- `existing_data = ...filter(employee_no3=employee_no, work_day2=work_day2)` … 「同じ従業員番号」かつ「同じ就業日」の既存データを探します。
- `if existing_data.exists():` … 既存があるなら、
- `existing_data.delete()` … 消します。これは「**ロードは上書き**」という仕様です。同じ人の同じ日のデータが二重に登録されるのを防ぎ、Excelの内容で置き換えます。

コピー

```
    # 新データをインスタンスとして作成してDBに保存
    try:
      # 工数テーブルに新しい行を作成・保存
      Business_Time_graph.objects.create(
        # 従業員番号
        employee_no3=employee_no,
        # 人員（先ほど取得したオブジェクトを紐付け）
        name=member_instance,
        # 3列目＝定義Ver
        def_ver2=ws.cell(row=i, column=3).value,
        # 就業日
        work_day2=work_day2,
        # 5列目＝直
        tyoku2=ws.cell(row=i, column=5).value,
        # 6列目＝作業内容
        time_work=ws.cell(row=i, column=6).value,
        # 7列目＝作業詳細
        detail_work=ws.cell(row=i, column=7).value,
        # 8列目＝残業時間
        over_time=ws.cell(row=i, column=8).value,
        # 9列目＝昼休憩
        breaktime=ws.cell(row=i, column=9).value,
        # 10列目＝残業休憩1
        breaktime_over1=ws.cell(row=i, column=10).value,
        # 11列目＝残業休憩2
        breaktime_over2=ws.cell(row=i, column=11).value,
        # 12列目＝残業休憩3
        breaktime_over3=ws.cell(row=i, column=12).value,
        # 13列目＝就業形態
        work_time=ws.cell(row=i, column=13).value,
        # 14列目＝入力OK_NG
        judgement=ws.cell(row=i, column=14).value,
        # 15列目＝休憩変更チェック
        break_change=ws.cell(row=i, column=15).value,
      )
    # 整合性エラー（重複等）が起きたら
    except IntegrityError as create_err:
      print(f"エラー: 行 {i} のデータ作成中に整合性エラーが発生しました: {create_err}。スキップします。")
      # 次のループへ進む            # その行を飛ばす
      continue
```

- `Business_Time_graph.objects.create(...)` … `.create(...)` は「新しい行を作って **即座にDBへ保存する**」命令。各引数が各列の値です。
- `name=member_instance` … ここだけ値ではなく **オブジェクト** を渡しています。先ほど `.get()` で取った人員を、外部キーとして紐付けます。
- `ws.cell(row=i, column=N).value` … i行目・N列目の値。**列番号と引数の対応** が、§6のヘッダー順と一致していることに注目してください。これがズレると意味が変わります。
- `except IntegrityError as create_err:` … 重複登録などDB制約に違反したら、ここで捕まえます。
- `continue` … その行だけ飛ばして次へ。1行の失敗で全体が止まらない設計です。

コピー

```
    # 5. 処理が成功したら一時ファイルを削除
    # 役目を終えた一時ファイルを削除
    os.remove(temp_file_path)
    # 成功を返す
    return {'status': 'success'}, None

  # ここまでで予期せぬエラーが出たら
  except Exception as e:
    # 一時ファイルが残っていれば
    if 'temp_file_path' in locals() and os.path.exists(temp_file_path):
      # 消す
      os.remove(temp_file_path)
    # エラーメッセージを返す
    return {'status': 'error', 'message': str(e)}, None
```

▶ ここで初めて出てくるコード

- **`locals()`**（Python）：その時の関数の中の変数を、名前と値の辞書で返します。`'temp_file_path' in locals()` で「その変数がもう作られているか」を調べられます。

- `os.remove(temp_file_path)` … 全行の処理が終わったら、一時ファイルを片付けます。サーバーにゴミを残さないためです。
- `return {'status': 'success'}, None` … 成功のタプルを返します。
- `except Exception as e:` … `try` ブロック全体で **想定外の例外** が起きたときの受け皿。`Exception` はあらゆるエラーの親なので、ここで一網打尽に捕まえます。
- `if 'temp_file_path' in locals() and os.path.exists(temp_file_path):` … `locals()` は「今この関数で定義済みの変数一覧」。`'temp_file_path' in locals()` で「その変数が作られているか」を確認し、かつファイルが残っていれば削除します。**エラーが早い段階で起きて変数が未定義の場合でも落ちない** ための丁寧な書き方です。
- `return {'status': 'error', 'message': str(e)}, None` … エラー内容（`str(e)`）を添えて返します。これがユーザー画面に表示されます。

なぜ一時ファイルを毎回消すのか？ アップロードされたExcelは「DBへ取り込むための一時的な置き場」に過ぎません。取り込みが終われば不要です。消さずに溜めると、サーバーのディスクが少しずつ埋まり、いずれ「容量不足」で全機能が止まります。だから **成功時も失敗時も必ず削除** しているのです。

9. tasks.py 全解説④：人員・班員・定義・その他のバックアップ

§6で「バックアップ関数はすべて同じ骨組み」と述べました。ここでは残りの関数を、**違いだけ** に注目して短く読みます。

9.1 人員データバックアップ `generate_member_backup`

> **▼ このコードがやること（先に日本語で）**：全人員のデータ（休憩時間設定やポップアップ設定を含む40列）をExcelに書き出します。

コピー

```
# 引数なし（全件対象だから）
def generate_member_backup():
  # 今日の日付を「20260602」形式の文字列に
  today = datetime.date.today().strftime('%Y%m%d')
  # ブック作成
  wb = openpyxl.Workbook()
  # シート取得
  ws = wb.active
```

- `def generate_member_backup():` … **引数なし**。工数バックアップは期間指定でしたが、人員は「全件」が対象なので日付を取りません。
- `today = datetime.date.today().strftime('%Y%m%d')` … `datetime.date.today()` で今日を取り、`.strftime('%Y%m%d')` で `20260602` のような文字列に整形します。`%Y` ＝4桁年、`%m` ＝2桁月、`%d` ＝2桁日。ファイル名に使います。

ヘッダーは40列（休憩時間設定6パターン×4・ポップアップ設定など）と多いですが、構造は §6 と同じ「`headers = [...]` → `ws.append(headers)`」です。データ書き込みも `member.objects.all()`（全人員）を `for` で回して `ws.append(row)` するだけ。保存も §6 と同一です。

> **用語： `strftime`（エスティーアールエフタイム：string format time）** 日付・時刻オブジェクトを、指定した書式の **文字列に変換** するメソッド。逆に文字列から日時を作るのは `strptime`（§9.6で登場）。`f` が共通で「format（書式）」を表します。

9.2 人員データロード `load_member_file`

工数ロード（§8）と骨組みは同じですが、1点だけ重要な違いがあります。

コピー

```
    for i in range(2, ws.max_row + 1):
      employee_no_value = ws.cell(row=i, column=1).value

      # 既に同じ番号の人員がいたら
      if member.objects.filter(employee_no=employee_no_value).exists():
        print(f"従業員番号 {employee_no_value} は既に存在するためスキップしました。")
        # 上書きせず飛ばす
        continue
```

- 工数ロード（§8）は「既存があれば削除して上書き」でしたが、人員ロードは「**既に存在すれば、上書きせずスキップ**」です。
- `if member.objects.filter(...).exists():` … 同じ従業員番号の人員が既にいるか確認し、いれば `continue` で飛ばします。
- **この違いは重要な仕様です。** 人員は「既存を壊さず、新しい人だけ追加する」方針。工数は「同じ日のデータを最新で置き換える」方針。**復元時の挙動が表ごとに違う** ことを必ず覚えてください。

9.3 班員データ `generate_team_backup` / `load_team_file`

- 班員（チーム構成）の表 `team_member` を扱います。1班に最大15人＋フォローON/OFFの17列。
- バックアップは §6 と同型。ロードも §9.2 と同じく「既存はスキップ」方式です。

9.4 工数区分定義 `generate_def_backup` / `load_def_file`

ここはヘッダーの作り方が独特なので注目します。

長い行を折り返しているので、なぞってコピーすると行が分かれます。正しくは →
コピー

```
  # ヘッダー作成
  headers = ['工数区分定義Ver名'] + [item for i in range(1, 51) for item in [f'工数区分名{i}', f'定義{i}',
f'作業内容{i}', f'変動/固定{i}']]
```

- `['工数区分定義Ver名'] + [...]` … 先頭に固定の1列を置き、その後ろに **50区分ぶんの列** を自動生成して連結します。
- `[item for i in range(1, 51) for item in [...]]` … **二重のリスト内包表記。** `i` を1〜50で回し、各 `i` について `工数区分名{i}`・`定義{i}`・`作業内容{i}`・`変動/固定{i}` の4列を作ります。結果、50×4＝200列＋先頭1列＝201列の見出しになります。手で201個書く代わりに、ループで自動生成しているわけです。

ロード `load_def_file` は「同じVer名の定義があれば削除してから登録」する上書き方式です。
`def_data[f'kosu_title_{n}'] = ...` のように列名を **動的に組み立てて** 辞書を作り、
`kosu_division(**def_data)` でまとめて生成しています。これも「200列を手で書かない工夫」です。

> **用語： `getattr(obj, '属性名')`** オブジェクトの属性を、**名前を文字列で指定して** 取り出す関数。`item.kosu_name` と書く代わりに `getattr(item, 'kosu_name')` と書けます。列名が `kosu_title_1` 〜 `kosu_title_50` のように番号付きで連番のとき、`getattr(item, f'kosu_title_{i}')` とループで一括処理できるので便利です（ `generate_def_backup` で使用）。

9.5 作業詳細選択肢・問い合わせ・管理者設定

- `generate_choice_backup` / `load_choice_file` … 作業詳細の選択肢（記号と詳細のペア）。ロードは「**全削除してから全登録**」する完全置き換え方式。
- `generate_inquiry_backup` / `load_inquiry_file` … 問い合わせデータ。ロードは人員を `.get()` で紐付けて `.create()`。すでにある問い合わせと同じかどうかは確かめずに、**全行を新しく足す**ので、同じファイルを 2 回ロードすると、同じ問い合わせが 2 件ずつになります。
- `generate_setting_backup` / `load_setting_file` … 管理者設定（表示件数・担当者・ポップアップ）。ロードは「**全削除してから登録**」方式。

> ⚠️ **ロードの「置き換え方」は表ごとに4種類** あります。混同すると復元時に事故ります。

方式	対象	挙動
上書き	工数・工数区分定義	同じキーの既存を消してから登録
スキップ	人員・班員	既存があれば触らず、新規だけ追加
全置換	作業詳細選択肢・管理者設定	テーブルを丸ごと消してから全件登録
追加だけ	問い合わせ	既存は確かめず、全行を新しく足す（2 回ロードすると 2 件ずつになる）

9.6 タスク履歴・操作履歴のバックアップ（タイムゾーン変換つき）

`generate_AsyncTask_backup` と `generate_History_backup` は、**時刻を日本時間に直して** から書き出すのが特徴です。

コピー

```
# 文字列を日付に変換
start_date = datetime.datetime.strptime(start_day, '%Y-%m-%d').date()
end_date = datetime.datetime.strptime(end_day, '%Y-%m-%d').date()
```

> **▶ ここで初めて出てくるコード**
>
> - **`.date()`**（Python）：日時（ `datetime` ）から、日付の部分だけを取り出します。

- `strptime(start_day, '%Y-%m-%d')` … `strftime`（§9.1）の逆。`'2026-05-01'` のような **文字列を日付オブジェクトに変換** します。`.date()` で時刻部分を落として日付だけにします。

コピー

```
for item in AsyncTask_data:
  try:
    # settings.TIME_ZONE（Asia/Tokyo）のタイムゾーン
    target_tz = pytz.timezone(settings.TIME_ZONE)
    # UTC→日本時間に変換しタイムゾーン情報を外す
    created_at_local = item.created_at.astimezone(target_tz).replace(tzinfo=None)
    updated_at_local = item.updated_at.astimezone(target_tz).replace(tzinfo=None)
  except ImportError:
    # 万一変換できなければそのまま
    created_at_local = item.created_at.replace(tzinfo=None)
    updated_at_local = item.updated_at.replace(tzinfo=None)
```

▶ ここで初めて出てくるコード

- `.timezone()`（Python）：`pytz.timezone('Asia/Tokyo')` は、時間帯（タイムゾーン）を表す物を作ります。日時を日本の時間に直す時に使います。
- `ImportError`（Python）：`import` に失敗した時（部品が入っていない時など）のエラー。`except ImportError:` で、無い時の代わりの処理を書けます。

- `target_tz = pytz.timezone(settings.TIME_ZONE)` … `settings.TIME_ZONE`（`'Asia/Tokyo'`）からタイムゾーンオブジェクトを作ります。
- `item.created_at.astimezone(target_tz)` … DBにはUTC（協定世界時）で保存されている時刻を、**日本時間（+9時間）に変換** します。
- `.replace(tzinfo=None)` … タイムゾーン情報（「これは日本時間です」というラベル）を **外します**。Excelはタイムゾーン付き時刻をうまく扱えないため、見た目だけ日本時間にして「素の時刻」として書き出すのです。

用語：UTC（ユーティーシー：協定世界時） 世界共通の基準時刻。日本時間はUTCより9時間進んでいます（UTC+9）。Djangoは `USE_TZ = True`（§11）の設定で、DBには **常にUTCで保存** します。表示やExcel出力のときに日本時間へ直すのが定石です。そうすると、海外からアクセスがあっても時刻のズレで混乱しません。

⚠️ **時刻のズレは運用トラブルの定番です。**「ログの時刻が9時間ずれている」という問い合わせは、たいていUTCと日本時間の混同です。本アプリはこの `astimezone` 変換でズレを吸収しています。

10. asynchronous_views.py 全解説：非同期フローとポーリング

ここまでは「作業の中身（tasks.py）」でした。次は、その作業を **いつ・どう起動するか** を司る `asynchronous_views.py` を読みます。これが §5 の流れ図の「③受付・スレッド起動」「⑤状態確認」「⑥配信」を担います。

10.1 冒頭インポート

コピー

```
# ファイル操作
import os
# 別スレッドを起動する道具（非同期の要）
import threading
# 一意なID（タスクID）を発行する道具
import uuid
# 日付処理（バリデーション用）
import datetime
# sleep（待ち）に使う
import time
# 一時ファイルを安全に作る道具
import tempfile
from ..tasks import generate_kosu_backup, delete_kosu_data, load_kosu_file, \
                    generate_member_backup, load_member_file, generate_team_backup, load_team_file, \
                    generate_def_backup, load_def_file, generate_choice_backup, load_choice_file, \
                    generate_inquiry_backup, load_inquiry_file, generate_setting_backup, load_setting_file, \
                    generate_AsyncTask_backup, delete_AsyncTask_data, generate_History_backup ,delete_History_data
# タスク状態を保存するモデル
from ..models import AsyncTask
# HTTPステータスコードの定数（status.HTTP_400_...）
from rest_framework import status
# API用デコレータ
from rest_framework.decorators import api_view, parser_classes
# リクエストの解析方式
from rest_framework.parsers import MultiPartParser, JSONParser, FormParser
# URLからurl_nameを逆引きする道具
from django.urls import resolve
# JSON応答・ファイル応答
from django.http import JsonResponse, FileResponse
```

- `import threading` … **非同期の心臓**。`threading.Thread(...)` で「別の流れ（スレッド）」を起動し、重い処理をそこで動かします。
- `import uuid` … Universally Unique IDentifier（世界で一意なID）。重複しない受付番号を作ります。
- `import tempfile` … アップロードファイルを安全に一時保存するための道具。
- `from ..tasks import ...` … §6〜§9の関数を全部取り込みます。`..tasks` の `..` は「1つ上の階層（`kosu/`）の `tasks.py`」を指します。
- `from ..models import AsyncTask` … タスクの状態（受付番号・成否・結果）を保存する表。
- `from rest_framework import status` … `status.HTTP_400_BAD_REQUEST`（＝400）のように、数字のステータスコードを読みやすい名前で使うための定数群。
- `api_view, parser_classes` … この関数を「DRF（Django REST Framework）のAPIとして扱う」「リクエストをどう解析するか指定する」ためのデコレータ。

- `resolve` … URLのパスから `url_name`（urls.pyで付けた名前）を逆算する道具。どのバックアップが要求されたかを判別するのに使います。
- `JsonResponse, FileResponse` … 前者はJSON（文字データ）を返す、後者はファイルそのものを返す応答クラス。

> **用語：デコレータ（decorator：装飾）** 関数の直前に `@名前` の形で付け、その関数に機能を「追加」するしくみ。`@api_view(['POST'])` は「この関数はPOSTリクエストを受けるAPIです」という意味を関数に着せます。プレゼントの包装紙のように、中身（関数）はそのままに外側を飾るイメージ。

10.2 受付関数 `backup`：タスクID発行とスレッド起動

コピー

```
# POSTリクエストのみ受け付けるAPI
@api_view(['POST'])
# ファイルもJSONもフォームも解析できる
@parser_classes([MultiPartParser, JSONParser, FormParser])
# リクエストを受け取る
def backup(request):
  # タスクID生成
  # 世界で一意なIDを生成して文字列に
  task_id = str(uuid.uuid4())
  # 「処理中」状態でタスクを記録
  AsyncTask.objects.create(task_id=task_id, status='pending')

  # リクエストから開始日を取得
  start_day = request.data.get('start_day')
  # 終了日を取得
  end_day = request.data.get('end_day')
```

- `@api_view(['POST'])` … この `backup` 関数は **POSTでのみ** 呼べます。GETでアクセスすると405エラー（§15）になります。
- `@parser_classes([...])` … 送られてくるデータが「ファイル付き（MultiPart）」「JSON」「フォーム」のいずれでも解析できるようにします。バックアップは日付JSON、ロードはファイル、と入力形式が違うため複数指定しています。
- `task_id = str(uuid.uuid4())` … `uuid.uuid4()` でランダムな一意IDを作り、`str()` で文字列化します。例：`'3f2a...e9'`。これが「受付番号」です。
- `AsyncTask.objects.create(task_id=task_id, status='pending')` … その受付番号を `status='pending'`（処理中）でDBに登録します。これが後でポーリングの参照先になります。
- `start_day = request.data.get('start_day')` … リクエストの中身から開始日を取り出します。`.get(...)` は「無ければ `None`」を返す安全な取り方。

コピー

```
# url_name属性取得
# 今アクセスされたURLのパス
current_path = request.path
# そのパスを解析
match = resolve(current_path)
# urls.pyで付けた名前を取り出す
url_name = match.url_name
```

- `request.path` … 今リクエストされたURLのパス部分（例：`/api/kosu/kosu_backup`）。
- `resolve(current_path)` … そのパスが、urls.py のどのルートに対応するかを解析します。
- `match.url_name` … urls.py で `name='kosu_backup'` のように付けた名前を取得。**この名前で「どの処理をすべきか」を分岐** します。1つの `backup` 関数が、URL名によって工数・人員・班員…と振る舞いを変えるのです。

コピー

```
# 工数バックアップなら
if url_name == 'kosu_backup':
  # 日付を検証
  error_response = validate_dates(start_day, end_day)
  # 検証エラーがあれば
  if error_response:
    # そのエラーを即返す
    return error_response
  # 実行する関数を「工数バックアップ」に決定
  task_function = generate_kosu_backup
  # その関数に渡す引数
  args = (start_day, end_day)
# 工数削除なら
elif url_name == 'kosu_delet':
  error_response = validate_dates(start_day, end_day)
  if error_response:
    return error_response
  #  「工数削除」関数に決定
  task_function = delete_kosu_data
  args = (start_day, end_day)
```

- `if url_name == 'kosu_backup':` … URL名で分岐。工数バックアップのときは…
- `error_response = validate_dates(start_day, end_day)` … §10.3の日付検証を呼びます。問題があればエラー応答が返ります。
- `if error_response: return error_response` … 検証でエラーが出ていたら、ここで処理を止めて返します。
- `task_function = generate_kosu_backup` … **関数そのものを変数に入れます**（カッコを付けない点に注意。`generate_kosu_backup()` だと「実行結果」、`generate_kosu_backup` だと「関数自体」）。後でスレッドに渡して実行します。

- `args = (start_day, end_day)` … その関数に渡す引数をタプルにまとめます。

> **用語：関数を変数に入れる（第一級関数）** Pythonでは関数も「値」として変数に代入できます。`f = generate_kosu_backup` とすると、`f(...)` で同じ関数を呼べます。本コードは「URL名に応じて、実行する関数を変数 `task_function` に選び、最後にまとめて起動する」設計で、これにより分岐ごとに同じ起動コードを書かずに済んでいます。

ファイルロード系の分岐は、アップロードファイルを一時保存します。

コピー

```
# 工数ロードなら
elif url_name == 'kosu_load':
  # アップロードされたファイルを取得
  kosu_file = request.FILES.get('file')
  # 一時パスを初期化
  temp_file_path = None
  try:
    # 一時ファイルを作成
    with tempfile.NamedTemporaryFile(suffix=".xlsx", delete=False) as temp_file:
      # アップロードを小分け（チャンク）で
      for chunk in kosu_file.chunks():
        # 一時ファイルに書き込む
        temp_file.write(chunk)
      # 一時ファイルのパスを記憶
      temp_file_path = temp_file.name
    # 実行関数を「工数ロード」に
    task_function = load_kosu_file
    # その一時パスを引数に
    args = (temp_file_path,)
  except Exception as e:
    # 失敗したら一時ファイルを
    if temp_file_path and os.path.exists(temp_file_path):
      # 消す
      os.remove(temp_file_path)
    return JsonResponse({'status': 'error', 'message': f'ファイル書き込みエラー: {str(e)}'}, status=500)
```

- `kosu_file = request.FILES.get('file')` … アップロードされたファイルを取り出します。`request.FILES` はファイル専用の入れ物。
- `with tempfile.NamedTemporaryFile(suffix=".xlsx", delete=False) as temp_file:` … 一時ファイルを作ります。`suffix=".xlsx"` で拡張子を付け、`delete=False` で「閉じても自動削除しない」（後で `tasks.py` 側が処理後に消すため）。
- `for chunk in kosu_file.chunks():` … 大きいファイルを **小分け（チャンク）** で読み、メモリを使いすぎずに書き込みます。
- `temp_file.write(chunk)` … 一時ファイルに書き込みます。
- `temp_file_path = temp_file.name` … その一時ファイルの実際のパスを記憶。これを `load_kosu_file`（§8）に渡します。
- `except` 節 … 書き込み失敗時は一時ファイルを消し、500エラー（サーバーエラー、§15）を返します。

この `elif` が `def_load`・`member_load`・`team_load`・`choice_load`・`inquiry_load`・`setting_load` と続きますが、**すべて同じ構造**（ファイル取得→一時保存→対応する `load_*` 関数を選択）です。引数のファイル変数

名と呼ぶ関数が違うだけです。

`AsyncTask_backup` などの履歴系は、再び `validate_dates` を通してから対応関数を選びます。最後に、選んだ関数を **別スレッドで起動** します。

コピー

```
else:
  return JsonResponse({'status': 'error', 'message': '無効なタスクタイプです。'}, status=status.HTTP_400_BAD_REQUEST)

# 別スレッドを用意
thread = threading.Thread(target=handle_task, args=(task_id, task_function, *args))
# スレッドを開始（裏で処理が走り出す）
thread.start()

# タスクIDを返却し、非同期処理開始を通知
# 受付番号を即座に返す
return JsonResponse({'status': 'success', 'task_id': task_id})
```

- `else: return JsonResponse(...)` … どのURL名にも当てはまらなければ「無効なタスクタイプ」として400エラー。
- `thread = threading.Thread(target=handle_task, args=(task_id, task_function, *args))` … **別スレッド** を用意します。`target=handle_task`（§10.6の関数）を、`args` の引数で動かします。`*args` は「タプルを展開して個別の引数として渡す」記法。
- `thread.start()` … スレッドを **開始**。この瞬間、重い処理は裏で走り始め、`backup` 関数自身はすぐ次の行へ進みます。
- `return JsonResponse({'status': 'success', 'task_id': task_id})` … **処理の完了を待たずに** 受付番号を即返します。これが非同期の本質です。ユーザーは番号を受け取り、後でポーリングで結果を取りに来ます。

> **なぜ django-q ではなく threading なのか（再掲）** §3.4 で触れたとおり、本アプリは設定上 `Q_CLUSTER` を持ちますが、バックアップは `threading.Thread` で動かしています。`threading` は「同じプロセス内で別の流れを作る」軽い方法で、`qcluster` ワーカーを別途起動しなくても動く利点があります。一方、サーバー再起動でスレッドが消える・複数台構成で扱いにくい等の弱点もあり、本格運用では django-q への移行が検討対象になります。「**今は threading で動いている**」という事実を運用者として押さえておきましょう。

10.3 日付バリデーション `validate_dates`

コピー

```
# 日付バリデーション関数
# 開始日・終了日を検証
def validate_dates(start_day, end_day):
  # 今日を文字列で
  today_str = datetime.date.today().strftime('%Y-%m-%d')
  # どちらか空なら
  if not start_day or not end_day:
    return JsonResponse({'status': 'error', 'message': '日付を指定してください。'}, status=status.HTTP_400_BAD_REQUEST)

  try:
    # 文字列を日付に
    end_date_obj = datetime.date.fromisoformat(end_day)
    today_date_obj = datetime.date.fromisoformat(today_str)
    start_date_obj = datetime.date.fromisoformat(start_day)
  # 変換できない＝形式不正なら
  except ValueError:
    return JsonResponse({'status': 'error', 'message': '日付の形式が不正です。'}, status=status.HTTP_400_BAD_REQUEST)
  # 終了日が今日以降なら
  if end_date_obj >= today_date_obj:
    return JsonResponse({'status': 'error', 'message': '昨日の日付までしか指定できません。'}, status=status.HTTP_400_BAD_REQUEST)
  # 開始日が終了日より後なら
  if start_date_obj > end_date_obj:
    return JsonResponse({'status': 'error', 'message': '開始日が終了日を超えています。'}, status=status.HTTP_400_BAD_REQUEST)
  # 問題なければ None（エラーなし）を返す
  return None
```

- `if not start_day or not end_day:` … 開始日か終了日が空（`None` や空文字）なら、「日付を指定してください」とエラー。
- `datetime.date.fromisoformat(end_day)` … `'2026-05-31'` のようなISO形式の文字列を日付オブジェクトに変換。形式が違うと `ValueError` が出て、`except` で「形式が不正」エラーになります。
- `if end_date_obj >= today_date_obj:` … 終了日が **今日以降** なら拒否。「**昨日まで** しか指定できない」という業務ルールです。
- `if start_date_obj > end_date_obj:` … 開始日が終了日より後ならおかしいので拒否。
- `return None` … 全部クリアなら `None`（エラーなし）。`backup` 関数側は「`None` なら問題なし」と判断して処理を続けます。

> **なぜ「昨日まで」なのか？** 当日のデータはまだ入力途中（更新され続けている）可能性が高いためです。確定していないデータをバックアップ・削除すると、入力途中の工数を巻き込む恐れがあります。「確定済み（昨日以前）」だけを対象にする安全策です。

10.4 状態確認 `check_task_status`（ポーリングの相手）

コピー

```
# GETで呼ぶAPI
@api_view(['GET'])
def check_task_status(request):
  # クエリ文字列からタスクIDを取得
  task_id = request.GET.get('task_id')

  # タスクIDがない場合、エラーを返す
  if not task_id:
    return JsonResponse({'status': 'error', 'message': 'タスクIDが指定されていません。'}, status=status.HTTP_400_BAD_REQUEST)

  try:
    # データベースからタスクIDに対応する状態を取得し返す
    # その受付番号のタスクを取得
    task = AsyncTask.objects.get(task_id=task_id)
    # 成功していれば
    if task.status == 'success':
      # 成功＋ファイルパスを返す
      return JsonResponse({'status': 'success', 'file_path': task.result})
    # 失敗していれば
    elif task.status == 'error':
      # エラーメッセージを返す
      return JsonResponse({'status': 'error', 'message': task.result})
    # まだ処理中なら
    else:
      # 「処理中(202)」を返す
      return JsonResponse({'status': 'pending'}, status=202)

  # その受付番号が存在しなければ
  except AsyncTask.DoesNotExist:
    return JsonResponse({'status': 'error', 'message': '無効なタスクIDです。'}, status=status.HTTP_400_BAD_REQUEST)
```

- `task_id = request.GET.get('task_id')` … URLのクエリ文字列（`?task_id=...`）から受付番号を取り出します。
- `task = AsyncTask.objects.get(task_id=task_id)` … その番号のタスクをDBから取得。
- `if task.status == 'success':` … 完了していれば、結果（ファイルパス）を返します。フロントはこれを使ってダウンロードへ進みます。
- `elif task.status == 'error':` … 失敗していれば、エラーメッセージを返します。
- `else: return JsonResponse({'status': 'pending'}, status=202)` … まだ `pending`（処理中）なら、**202** を返します。フロントは202を見て「まだだ。数秒後にもう一度聞こう」と再ポーリングします。
- `except AsyncTask.DoesNotExist:` … 番号が存在しなければ400エラー。

> **用語：202 Accepted（受理）**「リクエストは受け付けたが、処理はまだ完了していない」を表すHTTPステータスコード（§15）。非同期処理の「進行中」を伝えるのにぴったりのコードです。フロントは200（完了）でなく202を見て、ポーリングを続けます。

▼ ポーリングの実際の動き:



図9-3: ポーリング。画面が数秒おきに「終わった？」と聞き、200 と success が返ってきたらダウンロードへ進む

10.5 ファイル配信 `download_file`（後始末つき）

コピー

```
@api_view(['GET'])
def download_file(request):
  # 渡されたファイルパス
  file_path = request.GET.get('file_path')
  # そのファイルをバイナリ読み込みで開く
  file_handle = open(file_path, 'rb')
  # ダウンロード応答を作る
  response = FileResponse(file_handle, as_attachment=True, filename=os.path.basename(file_path))
```

- `file_path = request.GET.get('file_path')` … `check_task_status` が返したファイルパスを受け取ります。
- `file_handle = open(file_path, 'rb')` … ファイルを `'rb'`（read binary：バイナリ読み込み）で開きます。Excelは文字でなくバイナリなので `b` を付けます。
- `FileResponse(file_handle, as_attachment=True, filename=...)` … ファイルをそのまま返す応答。`as_attachment=True` で「ブラウザ内で開かず、ダウンロード（保存）させる」。`filename=os.path.basename(file_path)` で、保存時のファイル名をパスの末尾（ファイル名部分）にします。

コピー

```
# 遅延削除を行う内部関数
def delayed_file_cleanup():
    # 3秒待つ（ダウンロード完了を待つ）
    time.sleep(3)
    # ファイルがまだあれば
    if os.path.exists(file_path):
      try:
        # 削除
        os.remove(file_path)
      except Exception as e:
        # 失敗してもログだけ
        print(f"Cleanup failed after delay for {file_path}: {e}")

# 応答が閉じられたとき呼ばれる関数
def cleanup_on_close():
    # 削除を別スレッドで
    thread = threading.Thread(target=delayed_file_cleanup)
    thread.start()

# 応答の「閉じる」処理を差し替え
response.close = cleanup_on_close

# 応答を返す（ダウンロード開始）
return response
```

- `def delayed_file_cleanup():` … 「3秒待ってからファイルを消す」内部関数。`time.sleep(3)` で待つのは、ダウンロードが完了する前に消してしまわないための猶予です。
- `if os.path.exists(file_path): os.remove(file_path)` … ファイルがまだあれば削除。`try/except` で、消せなくてもログだけ出してプログラムは止めません。
- `def cleanup_on_close():` … 応答が閉じられた瞬間に呼ばれ、削除を **別スレッド** で起動します。応答を返す処理自体は止めずに、後始末を裏で行うためです。
- `response.close = cleanup_on_close` … 応答オブジェクトの「閉じる」動作を、自作の後始末関数に **差し替え** ます。
- `return response` … 応答を返すとダウンロードが始まり、終わると後始末が走ります。

> **なぜ「3秒後に削除」なのか？** バックアップファイルはユーザーがダウンロードした時点で役目を終えます。サーバーに残し続けるとディスクが埋まるので消したい。しかし、ダウンロード中に消すと壊れます。そこで「ダウンロードが終わるであろう数秒後」に消す折衷案です。**サーバーにファイルを溜めない** という運用上とても大事な工夫です。

10.6 汎用タスク処理 `handle_task`（スレッドの中身）

`backup` がスレッドで起動するのがこの関数です。タスク関数を実行し、結果をDBに書き戻します。

コピー

```
# 非同期タスク処理（汎用版）
# 受付番号・実行関数・引数を受け取る
def handle_task(task_id, task_function, *args, **kwargs):
  try:
    # タスク関数を実行し、結果を取得
    # 選ばれたタスク関数を実際に実行
    result = task_function(*args, **kwargs)
    # 戻り値が「明示的なエラー」かを判定
    is_explicit_error = (
      # 結果がタプルで
      isinstance(result, tuple) and
      # 要素があり
      len(result) > 0 and
      # 最初の要素が辞書で
      isinstance(result[0], dict) and
      # status が 'error' なら
      result[0].get('status') == 'error'
    )

    # 該当タスクを取得
    task = AsyncTask.objects.get(task_id=task_id)

    # 明示エラーの場合
    if is_explicit_error:
      error_dict = result[0]
      # 状態をエラーに
      task.status = 'error'
      # メッセージ保存
      task.result = error_dict.get('message', 'タスク関数が明示的なエラーを返しました。')
    # 正常終了の場合
    else:
      # 状態を成功に
      task.status = 'success'
      # 結果（ファイルパス）を保存
      task.result = result

    # データベースに保存
    # 変更をDBに書き込む
    task.save()

  # 予期せぬエラーが出たら
  except Exception as e:
    try:
      task = AsyncTask.objects.get(task_id=task_id)
      # 状態をエラーに
      task.status = 'error'
      # エラー内容を保存
      task.result = str(e)
      task.save()
    # タスク自体が無ければ
    except AsyncTask.DoesNotExist:
```

```
        # ログだけ
        print(f"Error: AsyncTask with id {task_id} not found.")
```

- `def handle_task(task_id, task_function, *args, **kwargs):` … 受付番号・実行する関数・その引数を受け取ります。`*args` は位置引数、`**kwargs` はキーワード引数をまとめて受ける記法。
- `result = task_function(*args, **kwargs)` … **ここで実際にタスクを実行**。例えば `generate_kosu_backup(start_day, end_day)` が走り、ファイルパスが `result` に入ります。
- `is_explicit_error = (...)` … 戻り値が「`({'status':'error', ...}, None)` のような明示エラーのタプル」かを4条件で判定します。`isinstance(x, 型)` は「xがその型か」を確認する関数。ロード処理（§8）はこの形でエラーを返すので、それを検出します。
- `task = AsyncTask.objects.get(task_id=task_id)` … 該当タスクをDBから取得。
- `if is_explicit_error:` … 明示エラーなら、状態を `'error'` にしてメッセージを保存。
- `else:` … 正常なら、状態を `'success'` にして結果（ファイルパス）を保存。
- `task.save()` … 変更をDBに書き込みます。**この瞬間に、ポーリング中のフロントが結果を取得できるようになります**（§10.4の `check_task_status` が読む先がここ）。
- `except Exception as e:` … タスク実行中に予期せぬ例外が出たら、状態をエラーにして内容を保存。さらに「タスク自体が見つからない」場合はログだけ出します（二重の安全策）。

> **2種類のエラーを区別している点に注目。**①ロード関数が「フォーマット不正」を **行儀よくタプルで返す** 明示エラー（`is_explicit_error`）と、② `openpyxl` が壊れたファイルで突然落ちるような **予期せぬ例外**（`except Exception`）。両方を捕まえて `status='error'` に統一し、フロントには一貫した形でエラーを伝えています。これが「どんな失敗でもユーザーにきちんとエラー表示が出る」堅牢さの正体です。

---

11. settings.py 全解説：本番設定の心臓部

`settings.py` は、アプリ全体の振る舞いを決める **設定の中枢** です。運用で触る機会が最も多いファイルの1つなので、1行ずつ読みます。

11.1 冒頭：パスと環境変数

コピー

```
# ファイルパスを扱う近代的な道具
from pathlib import Path
# OS機能（環境変数等）
import os
# .envファイルを読む外部ライブラリ
import environ
# ログを一定サイズで回転させるハンドラ
from logging.handlers import RotatingFileHandler

# プロジェクトの根っこフォルダのパス
BASE_DIR = Path(__file__).resolve().parent.parent

# 環境変数を読む道具を作成
env = environ.Env()
# ルートの.envファイルを読み込む
env.read_env(os.path.join(BASE_DIR, '.env'))

# 秘密鍵を.envから取得
SECRET_KEY=env('SECRET_KEY')

# デバッグモードを.envから真偽値で取得
DEBUG=env.bool('DEBUG')

# 既定の文字コードをUTF-8に
DEFAULT_CHARSET = 'utf-8'
```

- `from pathlib import Path` … パスを `/` 演算子などで直感的に扱える近代的なクラス。
- `BASE_DIR = Path(__file__).resolve().parent.parent` … `__file__`（このファイル `settings.py` の場所）から、`.parent.parent` で **2階層上**（プロジェクトの根っこ）を求めます。以降のパス指定の基準点です。
- `env = environ.Env()` … `.env` ファイルを読むための道具。
- `env.read_env(os.path.join(BASE_DIR, '.env'))` … ルートの `.env` を読み込みます。
- `SECRET_KEY=env('SECRET_KEY')` … `.env` から秘密鍵を取得。**Djangoが暗号署名（セッション・CSRF等）に使う最重要の鍵**。漏れると致命的なのでコードに直書きせず `.env` から読みます。
- `DEBUG=env.bool('DEBUG')` … デバッグモードを `True/False` で取得。**本番は必ず `False`**。`True` だとエラー時に内部情報がブラウザに丸見えになり危険です。
- `DEFAULT_CHARSET = 'utf-8'` … 文字コードをUTF-8に。日本語の文字化けを防ぎます。

⚠️ **本番で `DEBUG=True` は厳禁。**デバッグモードでは、エラーが起きたときにソースコードや設定値（時にDB接続情報まで）がブラウザに表示されます。攻撃者にヒントを与える重大なセキュリティ事故です。本番の `.env` は必ず `DEBUG=False` にします。

11.2 ALLOWED_HOSTS（アクセス許可ドメイン）

コピー

```
ALLOWED_HOSTS = [
    # 本番Azureのドメイン
    'hozen-kosu-react-cwaqashkafgbg5e5.japaneast-01.azurewebsites.net',
    # ローカル開発用
    'localhost',
    # ローカル開発用（IPアドレス表記）
    '127.0.0.1'
    ]
```

- `ALLOWED_HOSTS = [...]` …「このドメイン名で来たリクエストだけ受け付ける」というホワイトリスト。
- 1つ目はAzure本番のドメイン。2つ目・3つ目はローカル（自分のPC）。
- ここに無いドメインでアクセスされると、Djangoは `400 Bad Request` で拒否します。

なぜ必要？ これは「**Hostヘッダ攻撃**」を防ぐ仕組みです。攻撃者が偽のドメイン名でリクエストを送る攻撃を防ぐため、「正規のドメインのリストにあるものだけ受け付ける」のです。新しいドメインで公開するときは、ここに追加し忘れると「400 Bad Request」で全アクセスが弾かれます。**本番URLが変わったらここを更新** することを覚えておきましょう。

11.3 INSTALLED_APPS / MIDDLEWARE

コピー

```
INSTALLED_APPS = [
    # 管理画面
    'django.contrib.admin',
    # 認証
    'django.contrib.auth',
    # コンテンツタイプ
    'django.contrib.contenttypes',
    # セッション
    'django.contrib.sessions',
    # メッセージ
    'django.contrib.messages',
    # 静的ファイル
    'django.contrib.staticfiles',
    # Bootstrap4
    'bootstrap4',
    # 日付ピッカー
    'bootstrap_datepicker_plus',
    # 本アプリ本体
    'kosu',
    # 非同期タスクキュー
    'django_q',
    # REST API
    'rest_framework',
    # CORS（クロスオリジン）対応
    'corsheaders',
]
```

- `INSTALLED_APPS` … 「このプロジェクトで有効にする機能（アプリ）」の一覧。
- `'kosu'` … 本アプリ本体。`models.py` や `views/` がここに属します。
- `'django_q'` … §3.4 で触れた非同期タスクキュー。インストールはされています。
- `'rest_framework'` … API を作るためのDRF。
- `'corsheaders'` … 異なるドメイン間の通信（CORS）を許可する設定。

コピー

```
MIDDLEWARE = [
    # CORS処理（先頭に近い位置が必要）
    'corsheaders.middleware.CorsMiddleware',
    # セキュリティ
    'django.middleware.security.SecurityMiddleware',
    # セッション
    'django.contrib.sessions.middleware.SessionMiddleware',
    # 共通処理
    'django.middleware.common.CommonMiddleware',
    # CSRF対策
    'django.middleware.csrf.CsrfViewMiddleware',
    'django.contrib.auth.middleware.AuthenticationMiddleware',# 認証
    # メッセージ
    'django.contrib.messages.middleware.MessageMiddleware',
    # クリックジャッキング対策
    'django.middleware.clickjacking.XFrameOptionsMiddleware',
    # 静的ファイル配信
    'whitenoise.middleware.WhiteNoiseMiddleware',
    # 自作:現在リクエスト保持
    'kosu.middleware.clear_session_middleware.CurrentRequestMiddleware',
]
```

- `MIDDLEWARE` … リクエストとレスポンスが通る「処理のパイプライン」。上から順に通ります。
- `'whitenoise.middleware.WhiteNoiseMiddleware'` … **WhiteNoise**（ホワイトノイズ）。後述（§11.9）の通り、CSS・JS・画像などの静的ファイルを、専用サーバー無しでDjango自身が効率よく配信する仕組み。本番で重要。
- `'corsheaders.middleware.CorsMiddleware'` … CORS処理。リストの早い位置にあるのは、他の処理より前にCORSヘッダを付ける必要があるためです。
- 一番下は本アプリ自作のミドルウェア（現在のリクエストを保持する仕組み）。

> **用語：ミドルウェア（middleware：中間処理）** リクエストがビュー（処理本体）に届く前、そしてレスポンスがブラウザへ返る前に通る「関所」のような処理の列。セキュリティチェック・認証・静的ファイル配信などが各関所で行われます。
>
> **用語：WhiteNoise（ホワイトノイズ）** Djangoアプリが、CSS・JavaScript・画像といった「静的ファイル」を **自分で効率よく配信** できるようにするライブラリ。本来こうしたファイルはNginx等の専用サーバーが配るものですが、WhiteNoiseを使えばDjango単体（＋Gunicorn）で完結します。Azure App ServiceのようなPaaS環境では、これがあるおかげで構成がシンプルになります。

11.4 CORS設定

コピー

```
# すべてのオリジンを許可（開発向け）
CORS_ORIGIN_ALLOW_ALL = True
# Cookie等の資格情報を許可
CORS_ALLOW_CREDENTIALS = True
CORS_ALLOW_ORIGINS = [
    # React開発サーバー
    'http://localhost:3000',
    # Django開発サーバー
    'http://localhost:8000',
    'http://127.0.0.1:8000',
]
```

- `CORS_ORIGIN_ALLOW_ALL = True` … すべてのオリジン（ドメイン）からのAPI呼び出しを許可。開発では便利ですが、本番では絞るのが望ましい設定です。
- `CORS_ALLOW_CREDENTIALS = True` … Cookie（セッション認証）を異なるオリジン間でやり取りできるようにします。フロント（3000）とAPI（8000）が別ポートなので必要。

> **用語：CORS（コルス：Cross-Origin Resource Sharing）** ブラウザの安全機能で、「あるドメインのページが、別のドメインのAPIを呼んでよいか」を制御する仕組み。本アプリは開発時、フロント（ `localhost:3000` ）とバック（ `localhost:8000` ）が別オリジンなので、CORSの許可設定が必要になります。

11.5 REST_FRAMEWORK / セッション設定

コピー

```
REST_FRAMEWORK = {
    # ページ番号方式
    'DEFAULT_PAGINATION_CLASS': 'rest_framework.pagination.PageNumberPagination',
    # 1ページ20件
    'PAGE_SIZE': 20,
    'DEFAULT_PARSER_CLASSES': (
        # JSONを解析
        'rest_framework.parsers.JSONParser',
    ),
    'DEFAULT_RENDERER_CLASSES': (
        # JSONで返す
        'rest_framework.renderers.JSONRenderer',
    ),
}

# セッションをDBに保存
SESSION_ENGINE = 'django.contrib.sessions.backends.db'
# セッション有効期間（約10年=秒）
SESSION_COOKIE_AGE = 315360000

# Cookieを送る場面（'None'＝ほかのサイトからの通信でも送る）
SESSION_COOKIE_SAMESITE = 'None'
# HTTPS（暗号化した通信）の時だけCookieを送る
SESSION_COOKIE_SECURE = True
# ルーティングの起点
ROOT_URLCONF = 'hozen_another.urls'
```

- `'PAGE_SIZE': 20` … APIが一覧を返すとき、1ページあたり20件に区切ります。
- `SESSION_ENGINE = '...db'` … ログイン状態（セッション）を **データベースに保存** します。
- `SESSION_COOKIE_AGE = 315360000` … 約10年（秒数）。事実上「ログインしっぱなし」にする長い有効期間です。
- `SESSION_COOKIE_SAMESITE = 'None'` / `SESSION_COOKIE_SECURE = True` … Cookieの送信条件。`'None'` は「ほかのサイトから来た通信にも付けて送る」、`True` は「HTTPS の時だけ送る」。ブラウザは `'None'` を `Secure`（`True`）と組み合わせた時しか受け付けないので、2 つはセットです。これが GitHub の `master`（＝本番）の設定です。

> ⚠️ **手元で `'Lax'` / `False` に書き換えたら、コミットしない。** 手元の `http://localhost` で動かした時、ブラウザによっては `Secure` のクッキーが保存されず、ログインしてもすぐログイン画面に戻ることがあります。その時は手元でだけ `'Lax'` / `False` に書き換えて試しますが、そのままコミットして `master` に push すると、本番の設定まで変わります。`frontend/.env.production` の切り替え（01 章 §17.1）と合わせて、**「手元でだけ書き換えて、コミットしない 2 か所」**として覚えてください（「000 引き継ぎ書」の §8）。

11.6 DATABASES（データベース接続）

コピー

```
DATABASES = {
    'default': {
        # PostgreSQLを使う
        'ENGINE': 'django.db.backends.postgresql',
        # DB名（環境変数から）
        'NAME': os.getenv('DB_NAME', ''),
        # 接続ユーザー
        'USER': os.getenv('DB_USER', ''),
        # パスワード
        'PASSWORD': os.getenv('DB_PASSWORD', ''),
        # 接続先ホスト
        'HOST': os.getenv('DB_HOST', ''),
        # ポート番号（PostgreSQL既定）
        'PORT': '5432',
    }
}
```

- `'ENGINE': '...postgresql'` … データベースとして **PostgreSQL**（ポストグレスキューエル）を使う指定。
- `'NAME'/'USER'/'PASSWORD'/'HOST'` … `os.getenv('キー', '')` で **環境変数から** 取得。第2引数 `''` は「無ければ空文字」という既定値。これらは `.env`（ローカル）やAzureのアプリ設定（本番）に書かれます。
- `'PORT': '5432'` … PostgreSQLの標準ポート。

> **なぜ接続情報を環境変数に？** DBのパスワードをコードに直書きすると、GitHubに公開した瞬間に世界中に漏れます。環境変数（ `.env` ／Azure設定）に逃がすことで、**コードと秘密情報を分離** します。ローカルと本番で `DB_HOST` を変えるだけで接続先を切り替えられる利点もあります（§2.2）。
>
> ⚠️ 本番の `DB_HOST` は Azure Database for PostgreSQL のサーバー名（例： `xxx.postgres.database.azure.com` ）になります（§13）。ローカルでは `localhost` 。**この1つの環境変数で、つなぐDBが本番かローカルか決まる** ので扱いに注意します。

11.7 Q_CLUSTER（非同期タスク設定）

コピー

```
# Django Q の設定
Q_CLUSTER = {
    # タスク保存にdefaultのDBを使う
    'orm': 'default',
    # ワーカー（働き手）を4つ
    'workers': 4,
    # 2000タスクごとにワーカーを再生成
    'recycle': 2000,
    # リトライまでの秒数
    'retry': 2000,
    # タスクのタイムアウト（30分）
    'timeout': 1800,
}
```

- `Q_CLUSTER = {...}` … django-qの設定。§3.4で述べた通り、**現状バックアップは threading で動く** ため、この設定は直接は使われていませんが、将来のタスクキュー化のために定義されています。
- `'orm': 'default'` … タスクの管理情報を `default` のデータベース（§11.6）に保存します。
- `'workers': 4` … 同時に4つのタスクを処理できる。
- `'timeout': 1800` … 1タスクが30分（1800秒）を超えたら打ち切る。
- `'recycle': 2000` … メモリリーク対策で、2000タスクごとにワーカーを作り直す。

11.8 認証バリデータ・国際化

コピー

```
AUTH_PASSWORD_VALIDATORS = [
    # ユーザー情報と似たPW禁止
    {'NAME': 'django.contrib.auth.password_validation.UserAttributeSimilarityValidator'},
    # 最低長
    {'NAME': 'django.contrib.auth.password_validation.MinimumLengthValidator'},
    # よくあるPW禁止
    {'NAME': 'django.contrib.auth.password_validation.CommonPasswordValidator'},
    # 数字だけ禁止
    {'NAME': 'django.contrib.auth.password_validation.NumericPasswordValidator'},
]

# 表示言語を日本語に
LANGUAGE_CODE = 'ja'
# タイムゾーンを日本時間に
TIME_ZONE = 'Asia/Tokyo'
# 国際化を有効に
USE_I18N = True
# 時刻をタイムゾーン付きで扱う（DBはUTC保存）
USE_TZ = True
```

- `AUTH_PASSWORD_VALIDATORS` … パスワードの強度チェック群。短すぎ・ありがち・数字だけ、を拒否します。
- `LANGUAGE_CODE = 'ja'` … 管理画面などの言語を日本語に。
- `TIME_ZONE = 'Asia/Tokyo'` … §9.6 で使った `settings.TIME_ZONE` の正体。
- `USE_TZ = True` … **DBにはUTCで保存**し、表示時に日本時間へ直す方式（§9.6）。この設定があるから時刻処理が一貫します。

11.9 静的ファイル・WhiteNoise・CSRF

コピー

```
STATICFILES_DIRS = [
    # Reactビルドの静的ファイル置き場
    os.path.join(BASE_DIR, 'frontend/build/static'),
]

# Reactのbuild内のindex.htmlをテンプレートとして指定
# React のindex.htmlを探す場所
TEMPLATES[0]['DIRS'] = [os.path.join(BASE_DIR, 'frontend/build')]
```

- `STATICFILES_DIRS` … Djangoが静的ファイルを探す追加フォルダ。Reactのビルド成果物（`frontend/build/static`）を指定しています。
- `TEMPLATES[0]['DIRS'] = [...]` … Reactのビルドした `index.html` を、Djangoのテンプレートとして使う設定。**1つのDjangoが、APIもReactの画面も両方配信する** ための要です。

コピー

```
# 静的ファイルのURL接頭辞
STATIC_URL = '/static/'
# 主キーの既定型
DEFAULT_AUTO_FIELD = 'django.db.models.BigAutoField'
# collectstaticの集約先
STATIC_ROOT = os.path.join(BASE_DIR, 'staticfiles')

CSRF_TRUSTED_ORIGINS = [
  # 本番（HTTPS）
  'https://hozen-kosu-react-cwaqashkafgbg5e5.japaneast-01.azurewebsites.net',
  # ローカル
  'http://localhost:8000',
  'http://127.0.0.1:8000',
]

# WhiteNoiseの圧縮保存方式
STATICFILES_STORAGE = 'whitenoise.storage.CompressedManifestStaticFilesStorage'

# バックアップファイル保存先指定
# メディアファイルのURL接頭辞
MEDIA_URL = '/media/'
# バックアップExcelの保存先（tasks.pyで使用）
MEDIA_ROOT = os.path.join(BASE_DIR, 'media')
```

- `STATIC_URL = '/static/'` … ブラウザが静的ファイルを要求するときのURLの頭。
- `STATIC_ROOT = os.path.join(BASE_DIR, 'staticfiles')` … `collectstatic`（§3.5）が全静的ファイルを集める先。WhiteNoiseはここから配信します。
- `CSRF_TRUSTED_ORIGINS = [...]` … **CSRF対策で「信頼するオリジン」**。本番のHTTPS URLとローカルを列挙。ここに無いオリジンからのPOSTは、CSRF検証で弾かれます。**本番ドメインが変わったらここも更新** が必要です（`ALLOWED_HOSTS` とセットで覚える）。
- `STATICFILES_STORAGE = 'whitenoise.storage.CompressedManifestStaticFilesStorage'` … WhiteNoiseが静的ファイルを **gzip圧縮し、ファイル名にハッシュを付けて** 保存する方式。これにより通信量が減り、キャッシュも効きます（gitステータスに `.gz` ファイルが多いのはこのため）。
- `MEDIA_ROOT = os.path.join(BASE_DIR, 'media')` … §6〜§9の **`settings.MEDIA_ROOT` の正体**。バックアップExcelはこの `media/` フォルダに一時保存され、ダウンロード後に消されます。

用語：CSRF（シーエスアールエフ：Cross-Site Request Forgery）「ログイン中のユーザーになりすまして、悪意あるサイトが勝手にリクエストを送る」攻撃。Djangoはトークン（合言葉）でこれを防ぎます。

`CSRF_TRUSTED_ORIGINS` は「このオリジンからのPOSTは正規のものとして信頼する」リストです。

⚠️ **本番URLを変えるとき、3か所を必ず更新：** `ALLOWED_HOSTS`（§11.2）・`CSRF_TRUSTED_ORIGINS`（ここ）・フロントの `frontend/.env.production`（API接続先）。1つでも忘れると「アクセスできない」「POSTが403で弾かれる」といった障害になります。

11.10 LOGGING（ログ設定／RotatingFileHandler）

運用で最重要の1つ、**ログ設定** です。障害調査はログから始まります。

コピー

```
LOGGING = {
    # 設定スキーマのバージョン（必ず1）
    'version': 1,
    # 既存ロガーを無効化しない
    'disable_existing_loggers': False,
    # ログの書式定義
    'formatters': {
        'simple': {
            # メッセージ本文のみ出力
            'format': '{message}',
            # {}スタイルの書式
            'style': '{',
        },
    },
    # ログの出力先定義
    'handlers': {
        'web_console': {
            # INFO以上を記録
            'level': 'INFO',
            # サイズで回転するファイルハンドラ
            'class': 'logging.handlers.RotatingFileHandler',
            # 上のsimple書式を使う
            'formatter': 'simple',
            # 出力ファイル名
            'filename': 'web_console.log',
            # 最大ファイルサイズ（1MB）
            'maxBytes': 1024 * 1024 * 1,
            # 古いログを1世代だけ残す
            'backupCount': 1,
            # 文字コード
            'encoding': 'utf-8',
        },
    },
    # どのロガーがどのハンドラを使うか
    'loggers': {
        'views_logger': {
            # web_consoleハンドラに出す
            'handlers': ['web_console'],
            # INFO以上
            'level': 'INFO',
            # 親ロガーに伝播させない
            'propagate': False,
        },
    },
}
```

- `'version': 1` … Djangoのログ設定スキーマのバージョン。常に `1` です。

- `'disable_existing_loggers': False` … 既存（Django本体など）のロガーを無効化しない。
- `'formatters'` の `'simple'` … ログの書式。`'{message}'` で「メッセージ本文だけ」を出します（時刻やレベルは付けない最小構成）。
- `'handlers'` の `'web_console'` … **出力先の定義**。最重要は `'class': 'logging.handlers.RotatingFileHandler'`。
- `'class': '...RotatingFileHandler'` … **回転するファイルハンドラ**。ログファイルが大きくなりすぎないよう、一定サイズで自動的に切り替えます。
- `'filename': 'web_console.log'` … 出力ファイル名。プロジェクト直下に作られます。
- `'maxBytes': 1024 * 1024 * 1` … 1MB（1024×1024×1バイト）。ファイルがこのサイズに達すると回転します。
- `'backupCount': 1` … 古いログを **1世代だけ** 残す（`web_console.log.1`）。それ以上古いものは捨てる。
- `'encoding': 'utf-8'` … 日本語ログの文字化けを防ぐ。
- `'loggers'` の `'views_logger'` … コード内で `logging.getLogger('views_logger')` と呼ぶとこのロガーが使われ、`web_console` ハンドラ（上記ファイル）へ書き込まれます。
- `'propagate': False` … 親ロガーへ重複して流さない設定。

> **用語：RotatingFileHandler（ローテーティング・ファイル・ハンドラ）** ログファイルを「一定サイズに達したら新しいファイルに切り替え、古いものは決められた世代数だけ残して捨てる」仕組み。日記帳が1冊埋まったら新しい1冊に移り、古い1冊だけ取っておくイメージ。**ログでディスクが満杯になる事故** を防ぎます。本アプリは「1MB×2世代（現行＋1）」なので、ログは最大でも約2MBに収まります。
>
> ⚠️ **ログのサイズ制限はトレードオフ。** 1MB×2世代だと、アクセスが多い日には「古いログがすぐ消えて、障害発生時のログが残っていない」ことがあります。重要な障害を追うときは、ログが上書きされる前に `web_console.log` を退避（コピー）しておくのが運用のコツです。
>
> **ログレベル（重要度）の階段:**

コピー

```
DEBUG < INFO < WARNING < ERROR < CRITICAL
```

> 本アプリは `INFO` 以上を記録します。つまり `DEBUG`（詳細すぎる開発用）は記録せず、`INFO`（通常の動作記録）・`WARNING`（警告）・`ERROR`（エラー）・`CRITICAL`（致命的）を残します。

---

12. wsgi.py 全解説：Webサーバーとの玄関

`wsgi.py` は、**WebサーバーとDjangoアプリをつなぐ「玄関口」** です。短いファイルですが、本番で必ず使われる重要なファイルです。

用語：WSGI（ウィズギー：Web Server Gateway Interface）「WebサーバーとPythonアプリの間の、標準的な会話の作法（約束事）」。これがあるおかげで、Gunicorn（§13）のようなWebサーバーは、Djangoでもその他のPythonフレームワークでも、同じ作法で呼び出せます。電源コンセントの規格（差し込み口の形）が統一されているから、どのメーカーの家電も挿せるのと同じイメージ。

▼ このコードがやること（先に日本語で）：「DjangoのどのプロジェクトのWebアプリを動かすか」を指定し、Webサーバー（Gunicorn）が呼び出す `application` という名前の入口を用意します。

コピー

```
"""
WSGI config for hozen project.

It exposes the WSGI callable as a module-level variable named ``application``.

For more information on this file, see
https://docs.djangoproject.com/en/5.0/howto/deployment/wsgi/
"""

# 環境変数操作のため
import os

# DjangoのWSGIアプリを取得する関数
from django.core.wsgi import get_wsgi_application

# 設定ファイルの場所を指定
os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'hozen_another.settings')

# WSGIアプリ本体を作る（Gunicornが呼ぶ入口）
application = get_wsgi_application()
```

▶ ここで初めて出てくるコード

- **`.setdefault()`**（Python）：`os.environ.setdefault('名前', '値')` は「その環境変数がまだ無い時だけ、この値にする」。`DJANGO_SETTINGS_MODULE` で、どの設定ファイルを使うかを決めます。
- **`get_wsgi_application()`**（Django）：Django のアプリを、本番の Web サーバー（Gunicorn など）が呼べる形（WSGI）にして返します。`wsgi.py` の中で使います。
- **`"""…"""`**（Python）：`"""…"""`（`"` を 3 つ）で囲んだ文字は、何行にもまたがって書けます。関数・クラス・ファイルのいちばん最初に書いた物は、その説明書（docstring）になり、`help()` やエディターで読めます。
- **`.environ`**（Python）：`os.environ` は、環境変数の入れ物（辞書のような物）です。`os.environ.get("名前", 無い時の値)` で読みます。

- 冒頭の `"""..."""` … **docstring（ドキュメンテーション文字列）**。このファイルの説明と、公式ドキュメントへのリンクです。

- `import os` … 環境変数を設定するために使います。
- `from django.core.wsgi import get_wsgi_application` … DjangoのWSGIアプリを組み立てる関数を取り込みます。
- `os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'hozen_another.settings')` … **どの設定ファイルを使うか** を環境変数で指定します。`'hozen_another.settings'` は §11 の `settings.py`。`setdefault` は「すでに設定されていなければ設定する」（既存値を尊重する）優しい指定。
- `application = get_wsgi_application()` … **本番の入口**。Gunicorn（§13）はこの `application` という名前の変数を呼び出してリクエストを処理させます。この1行が「Webサーバーとアプリの接続点」です。

> **なぜ `application` という名前なのか？** WSGIの約束事で、Webサーバーは「`application` という名前の呼び出し可能オブジェクト」を探します。Gunicornを起動するとき `gunicorn hozen_another.wsgi:application` のように指定するのは、「**`hozen_another/wsgi.py` の中の `application` を使え**」という意味です（§13）。
>
> ⚠️ **`wsgi.py` は基本的に編集しません。** ここはDjangoの定型ファイルで、設定ファイル名（`hozen_another.settings`）だけが本アプリ固有です。触る機会はほぼありませんが、「本番の入口はここ」という知識は障害対応で役立ちます。

---

13. 本番アーキテクチャ：Azure 構成図

ここまでで「コードが何をするか」を理解しました。次は「それが本番で **どこで・どう動いているか**」を見ます。本アプリの本番は **Microsoft Azure（マイクロソフト・アジュール）** というクラウド上にあります。

> **用語：クラウド（cloud）** 自社でサーバー機械を買って置く代わりに、インターネット越しに他社（ここではMicrosoft）のコンピュータを借りて使う方式。電気や水道のように「使った分だけ払う」イメージ。Azureはその大手の1つです。

13.1 構成図（本番の全体像）

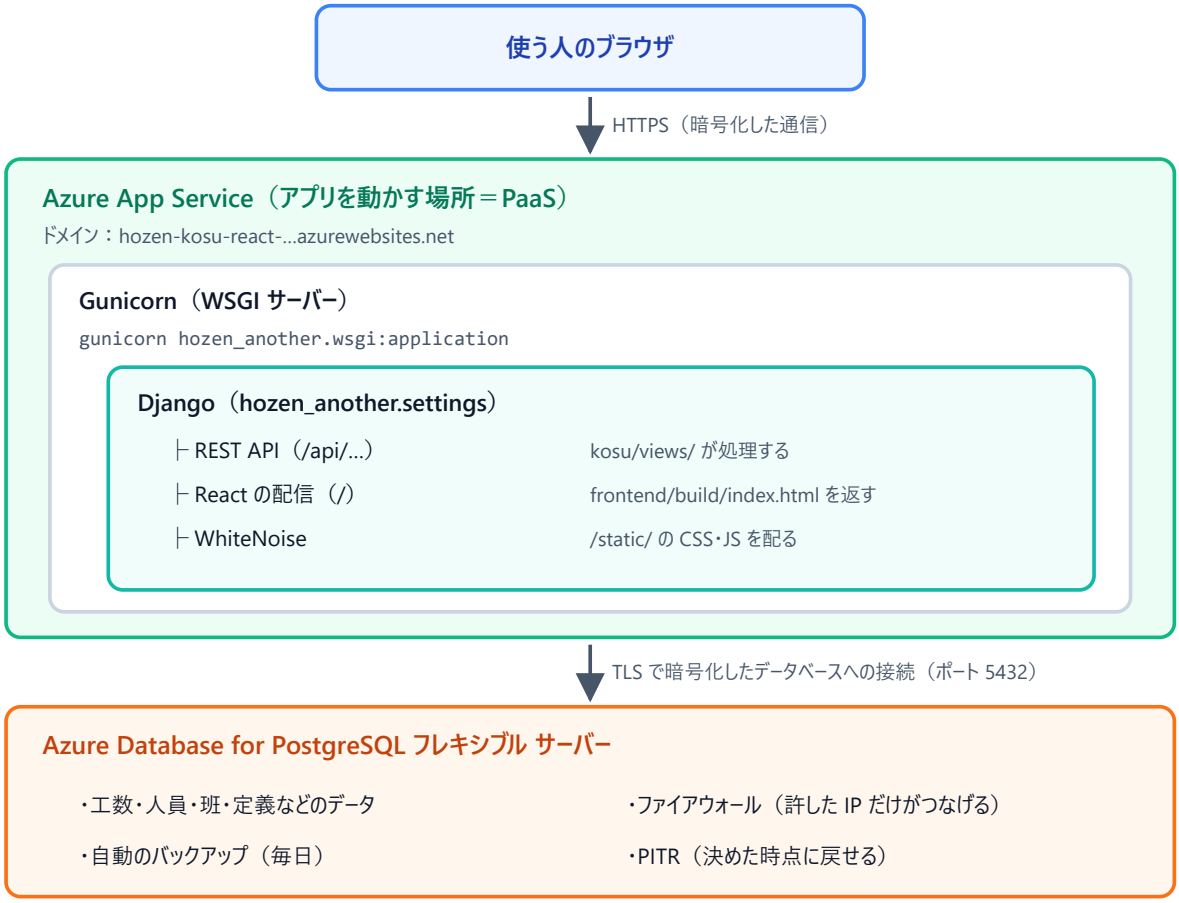


図9-4: 本番の環境のくわしい図。Azure App Service の中で Gunicorn が Django を動かし、データは Azure Database for PostgreSQL に保存する

13.2 各部品の役割

部品	よみ	役割
Azure App Service	アジュール・アップサービス	アプリを動かす実行環境（PaaS）。OSやサーバー管理をAzureが代行
Gunicorn	ガニコーン	Pythonアプリを本番で動かすWSGIサーバー。`wsgi.py` の `application` を呼ぶ
WhiteNoise	ホワイトノイズ	CSS・JS・画像（静的ファイル）を配信（§11.3）
Azure Database for PostgreSQL	—	データを保存するクラウドDB（フレキシブルサーバー）

> **用語：PaaS（パース：Platform as a Service）**「アプリを動かす土台（プラットフォーム）」をサービスとして借りる方式。サーバーの購入・OSのインストール・セキュリティパッチ当てなどをすべてAzureが代行し、利用者はアプリを置くだけで動きます。Azure App Service がこれにあたります。後任者が「サーバーのOSを管理する」必要がほぼ無いのが利点です。
>
> **用語：Gunicorn（ガニコーン：Green Unicorn）** Pythonの本番用WSGIサーバー。`runserver`（開発用）と違い、複数のリクエストを同時に・高速に・安定して処理できます。`gunicorn hozen_another.wsgi:application` のように、§12の `application` を指定して起動します。

13.3 Azure Database for PostgreSQL フレキシブルサーバーの要点

> **用語：フレキシブルサーバー（Flexible Server）** Azureが提供するPostgreSQLの運用形態の1つ。柔軟な設定（停止/起動・メンテナンス時間帯の指定など）ができ、自動バックアップやPITRを備えています。

運用者として押さえるべき4つの機能：

機能	意味	運用での意義
接続	`DB_HOST` に `xxx.postgres.database.azure.com` を指定（§11.6）	この値が本番DBへの入口
ファイアウォール	許可したIPアドレスからのみ接続可	不正アクセスを防ぐ。新しい場所から接続するならIP許可が必要
TLS	DB接続を暗号化	パスワードやデータが盗み見られない
自動バックアップ／PITR	毎日自動でバックアップ。任意の時点に戻せる	事故時の最後の砦

用語：ファイアウォール（firewall：防火壁）「許可した相手からの通信だけ通し、それ以外を遮断する」仕組み。Azure DBのファイアウォールは「このIPアドレスからの接続だけ許可」と設定します。**新しいオフィスや自宅から本番DBに接続できないときは、たいていIPが許可リストに無いのが原因** です。

用語：TLS（ティーエルエス：Transport Layer Security） 通信を暗号化する技術。ブラウザのHTTPSにも使われています。アプリとDBの間の通信をTLSで暗号化すると、途中で盗み見されてもパスワードやデータが読めません。

用語：PITR（ピットアール：Point-In-Time Restore）「特定の時点」にデータベースを巻き戻す機能。例えば「今日の14時30分に誤って大量削除した」場合、14時29分の状態に戻せます。Azureが継続的に変更を記録しているから可能です。**§6〜§9のアプリ内バックアップとは別の、より強力な最終手段** です。

⚠️ **2種類のバックアップを混同しない。**

種類	誰が	何を	戻し方
アプリ内バックアップ（§6〜§9）	運用者が手動	工数・人員などをExcelに	画面からロード
Azure自動バックアップ／PITR	Azureが自動	DB全体	Azureポータルで時点復元

前者は「特定の表を手元に保存」、後者は「DB全体を時間で巻き戻す」。役割が違うので、両方を理解しておきます。

---

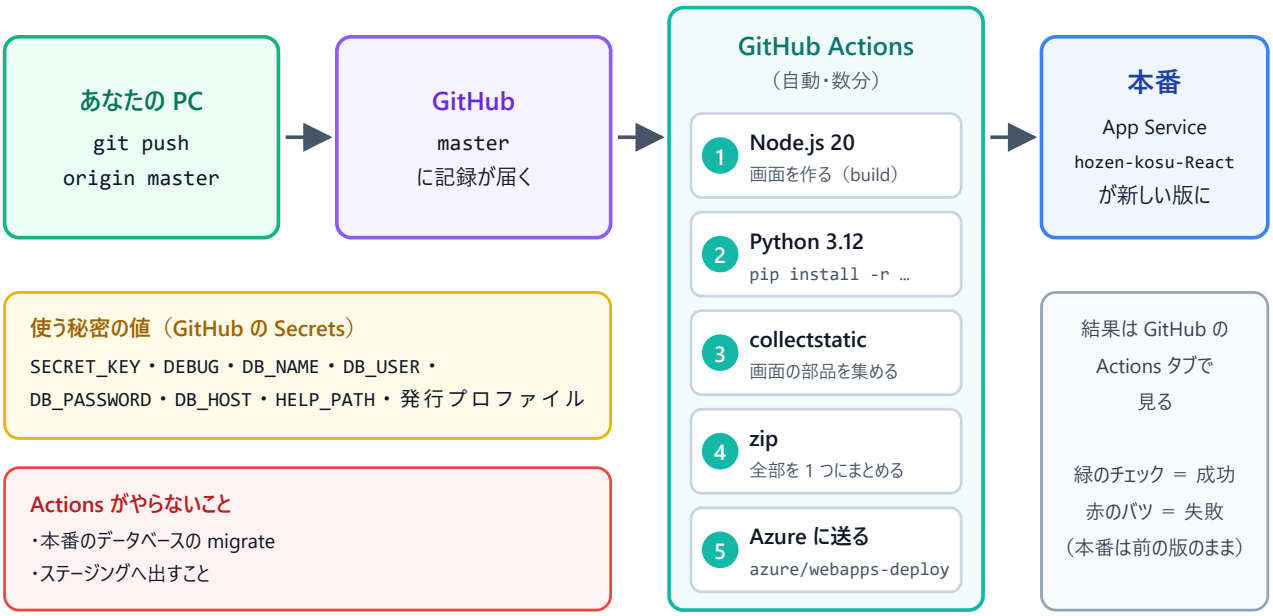
14. デプロイ手順と本番マイグレーションの注意

新しい版を本番へ出す「デプロイ」の手順です。まず「本番に出る仕組み」を知り、次に毎回の手順を見ます。**順番が命** なので、1つずつ確認しながら進めます。

14.1 本番に出る仕組み：master に push すると、GitHub Actions が出す

本アプリは、**GitHub の `master` ブランチに push した瞬間に、本番へ出ていきます。**だれかがボタンを押す必要はありません。仕組みを決めているのは、プロジェクト直下の `.github/workflows/master_hozen-kosu-react.yml`（GitHub Actions の設定ファイル）です。

用語：GitHub Actions（ギットハブ・アクションズ） GitHub に「push されたら、このコマンドを順に動かして」と頼んでおける仕組み。決めたコマンドは、GitHub が用意したコンピューター（ここでは Ubuntu という Linux）の上で動きます。本アプリでは「画面を作る → 部品を入れる → 集める → まとめる → Azure に送る」を自動でやらせています。



master に push した時のほか、Actions タブの「Run workflow」でも同じことが起きる

図9-6: 本番に出る流れ（.github/workflows/master_hozen-kosu-react.yml）。master に push すると、GitHub Actions が ①〜⑤ を自動で行う

設定ファイルに書いてあることを、日本語にすると次のとおりです。

順番	設定ファイルの名前（`name:`）	やること
合図	`on: push: branches: - master` と `workflow_dispatch`	`master` に push された時と、GitHub の **Actions** タブで「Run workflow」を押した時に動く
①	Set up Node.js / Install React dependencies / Build React	Node.js **20** を入れ、`frontend/` で `npm install` と `npm run build` をする（`frontend/.env.production` の本番の URL が画面に書き込まれる）
②	Set up Python / Install Python dependencies	Python **3.12** を入れ、`antenv` という仮想環境を作って `pip install -r requirements.txt`
③	Collect static files	`python manage.py collectstatic --noinput` で、画面の部品を `staticfiles/` に集める
④	Create deployment zip	`.git`・`node_modules`・`frontend/node_modules` を除いた全部を `release.zip` にまとめる
⑤	Deploy to Azure Web App	`azure/webapps-deploy@v3` で、Azure の App Service **`hozen-kosu-React`** に送る

各ステップの意味と注意：

- **画面を作るのも、部品を集めるのも、Actions がやる。**以前は「手元で `npm run build` と `collectstatic` をしてからコミット」と案内していましたが、今の仕組みでは Actions が毎回作り直すので、手元でやってコミットする必要はありません（`frontend/build/` と `staticfiles/` はリポジトリに入っていますが、本番に送られるのは Actions が作り直した物です）。
- **秘密の値は GitHub の Secrets から使う。**設定ファイルは `SECRET_KEY`・`DEBUG`・`DB_NAME`・`DB_USER`・`DB_PASSWORD`・`DB_HOST`・`HELP_PATH` と、Azure に送るための「発行プロファイル」（`AZUREAPPSERVICE_PUBLISHPROFILE_…`）を、GitHub の Secrets から読みます。置き場所は、GitHub のリポジトリの **Settings** → **Secrets and variables** → **Actions** です。Secrets は、一度保存すると値を見られません（上書きはできます）。
- **本番で動いているアプリが読む値は、Azure 側にあるはず。**リポジトリに `.env` は無く（§2.2）、Secrets は Actions が動いている間だけ使われます。そのため本番のアプリは、Azure の App Service の「環境変数」（以前の画面では「構成」→「アプリケーション設定」）から `SECRET_KEY` などを読んでいるはずです。リポジトリからは見えないので、Azure ポータルで確かめてください。
- **migrate はしない。**設定ファイルに `python manage.py migrate` はありません。モデルを変えて本番のデータベースの形を変える時は、別に行います（§14.3）。
- **結果は Actions タブで見る。**GitHub のリポジトリの **Actions** タブに、push のたびに 1 行ずつ実行の記録が増えます。緑のチェックなら成功、赤のバツなら失敗です。失敗した時は、本番は前の版のまま動き続けます（どのステップで失敗したかは、その行を開くと分かります）。
- **起動のコマンドは Azure 側の設定。**本番で Django を動かす Gunicorn（§13.2）を、どんなコマンドで起動しているかも、リポジトリには書いてありません。Azure ポータルの App Service の設定で確かめます。

> ⚠️ **`master` への push ＝ 本番の作業。**練習のつもりで `master` に push しても、本番が変わります。業務時間中に大きな変更を出すのは避け、出したら Actions が緑になるのを見届け、本番の画面で確かめるまでが 1 回のデプロイです。

14.2 標準デプロイ手順（毎回これ）

> **▼ この手順がやること（先に日本語で）**：手元でテストを通し、「手元でだけ書き換えた所」が戻っているのを確かめてから、コミットして `master` に push します。あとは GitHub Actions が本番に出すので、その結果と本番の画面を確かめます。

コピー

```
① 手元でテストを通す
   cd frontend
   npm run test:run            # フロントのテストが全部緑か確認
   cd ..
   python manage.py test --settings=hozen_another.test_settings   # バックのテスト（12章）

② 「手元でだけ書き換えた 2 か所」が戻っているか確かめる
   git status                  # 次の 2 つが modified なら、git diff で見て戻す
                               #   frontend/.env.production
                               #   hozen_another/settings.py

③ 変更をコミットする
   git add （変えたファイル）
   git commit -m "（変更内容）"

④ master に push する（＝本番に出る）
   git push origin master

⑤ GitHub の Actions タブで、緑のチェックになるまで見守る（数分）

⑥ モデルを変えた時だけ：本番のデータベースに migrate する（§14.3）

⑦ 本番の画面で確かめる（§14.4）
```

各ステップの注意：

- ① **テスト** … 壊れたコードを本番に出さない最後の防衛線。`test:run` が赤なら、push してはいけません。
- ② **手元でだけ書き換えた所** … `frontend/.env.production`（01章 §17.1）と `hozen_another/settings.py` のクッキーの 2 行（§11.5）は、手元で試す時に書き換えることがあります。戻し忘れて push すると、本番の画面がだれにも使えなくなったり、本番の設定が弱くなったりします。
- ③ **コミット** … `git add -A`（全部）より、変えたファイルを名前で指定するほうが、うっかり混ざるのを防げます。
- ④ **push** … これで本番に出ます（§14.1）。
- ⑤ **Actions** … 赤のバツなら、その行を開いて、どのステップで失敗したかを見ます。本番は前の版のまま動いているので、落ち着いて直して、もう一度 push します。
- ⑥ **マイグレーション** … モデルを変えていないなら不要。変えたなら **慎重に**（次項）。

> ⚠️ **「コードは直したのに本番が変わらない」時は、まず Actions タブを見る。** 赤のバツ（失敗）になっていないか、そもそも `master` に push できているか（`git log origin/master` に自分のコミットがあるか）を確かめます。画面が古いままの時は、ブラウザに古い画面が残っていることもあるので、`Ctrl` + `F5`（強制的に読み直す）も試します。

14.3 本番マイグレーションの注意

本番DBへのマイグレーションは、**最も事故が起きやすい操作** です。次を厳守します。

> ⚠️ **本番マイグレーション5つの鉄則** 1. **必ず事前にバックアップを取る** — アプリ内バックアップ（§6）＋Azure自動バックアップの両方を確認。2. **列・テーブルの削除は特に危険** — 削除マイグレーションは既存データを永久に失う。本当に必要か二重確認。3. **ユーザーが少ない時間帯に行う** — 業務時間中は避ける。マイグレーション中は不整合が起きうる。4. **ローカル／ステージングで先に試す** — 同じマイグレーションを本番前に必ず別環境で実行し、エラーが出ないか確認。5. **`migrate` の前に `--plan` で確認** — `python manage.py migrate --plan` で「何が実行されるか」を事前に見る。
>
> **なぜ削除が危険なのか？** 例えば `member` テーブルから列を削除するマイグレーションを本番で実行すると、その列に入っていた **全ユーザーのデータが消えます**。`makemigrations` が自動生成した削除を、内容を確認せずに `migrate` してしまうのが典型的な事故です。生成されたマイグレーションファイル（ `kosu/migrations/` ）の中身を **必ず読んでから** 適用します。
>
> **用語： `migrate --plan`** 実際には適用せず、「これから何のマイグレーションが、どの順で実行されるか」の一覧だけを表示するオプション。本番適用前の「最終確認」に使います。（※ `--plan` はDjangoの標準オプション）
>
> ⚠️ **本番で migrate を打つ場所は、前任者に確かめる。**§14.1 のとおり、GitHub Actions は migrate をしません。本番のデータベースに migrate するには、本番のデータベースにつながる所で `python manage.py migrate` を打つ必要があります（例えば、Azure ポータルの App Service の「SSH」から打つ、本番の接続先を入れた手元の環境から打つ、など）。前任者がどの方法で行っていたかはリポジトリに書かれていないので、必ず確かめてから行ってください（「000 引き継ぎ書」の §10）。

14.4 デプロイ後の確認

デプロイしたら、必ず本番で動作確認します。

コピー

```
□ トップページが表示されるか（500/502エラーが出ないか）
□ ログインできるか
□ 主要機能（工数入力・一覧表示）が動くか
□ 今回変更した機能が意図どおり動くか
□ web_console.log にエラーが出ていないか（§11.10）
```

> ⚠️ **デプロイ直後の数分は特に注意。**502 Bad Gateway（§15）が出る場合、Gunicornの起動に失敗している（環境変数未設定・依存ライブラリ不足など）可能性があります。ログを確認してください。

---

15. HTTPステータスコード早見表

運用・障害対応で必ず出会う **HTTPステータスコード** を、本アプリでの意味とともにまとめます。

> **用語：HTTPステータスコード** サーバーがブラウザに返す「3桁の数字の返事」。「うまくいった」「あなたの送り方が悪い」「サーバー側が壊れた」などを数字で伝えます。百の位で大まかな種類が分かります。

百の位	種類	意味
2xx	成功	リクエストは成功した
3xx	リダイレクト	別の場所へ移動して
4xx	クライアントエラー	あなた（送り手）の送り方が問題
5xx	サーバーエラー	サーバー側が壊れた

主要コードと本アプリでの登場場面：

コード	名前	意味	本アプリでの例
200	OK	成功	API正常応答・ダウンロード成功
202	Accepted	受理（処理中）	`check_task_status` のpending（§10.4）
301/302	Moved/Found	リダイレクト	401時に `/login` へ誘導（CLAUDE.md）
400	Bad Request	リクエストが不正	`validate_dates` の日付エラー（§10.3）、`ALLOWED_HOSTS` 外
401	Unauthorized	未認証	未ログイン。`/login` へリダイレクト
403	Forbidden	権限なし／CSRF不正	`CSRF_TRUSTED_ORIGINS` 外からのPOST（§11.9）
404	Not Found	見つからない	存在しないURL・ファイル
405	Method Not Allowed	許可されないメソッド	GETで `backup`（POST専用）を叩く（§10.2）
500	Internal Server Error	サーバー内部エラー	コードのバグ・例外。ファイル書き込み失敗（§10.2）
502	Bad Gateway	ゲートウェイ不正	Gunicorn起動失敗・クラッシュ（§14.4）
503	Service Unavailable	一時的に利用不可	サーバー過負荷・再起動中

> ⚠️ **400と500の切り分けが障害対応の第一歩。**4xx（特に400/401/403）は「**リクエスト側の問題**」＝設定や送り方が原因。5xx（500/502）は「**サーバー側の問題**」＝コードのバグやサーバー障害。まずこの大分類で原因の方向を絞ります。
>
> **▼ 障害コードの読み解き例:** - 「403が出る」→ CSRFトークンの問題か、`CSRF_TRUSTED_ORIGINS`（§11.9）にオリジンが無い。- 「400 Bad Request」→ `ALLOWED_HOSTS`（§11.2）にドメインが無い、または日付バリデーションエラー。- 「502 Bad Gateway」→ Gunicornが起動できていない。環境変数・依存ライブラリ・`wsgi.py` を確認。- 「500」→ コードのバグ。`web_console.log` とAzureのログでスタックトレースを確認。

---

16. 障害対応フロー

実際に「動かない！」という連絡が来たときの、落ち着いた対応手順です。

16.1 障害対応の基本ステップ

> **▼ この手順がやること（先に日本語で）:** あわてず、症状を確認し、範囲を切り分け、ログを見て、原因を特定し、対処し、記録する、という順で進めます。



あわてて直し始めず、①から順に進める

図9-5: 障害が起きた時の 6 つの手順。くわしい中身は、図の下の箇条書きにある

1. **症状を正確に把握する** - だれが・いつ・何をしたら・どうなったか - エラーメッセージ・ステータスコード（§15）を聞く／見る - スクリーンショットをもらう
2. **範囲を切り分ける** - 全員で起きる？ 特定のユーザーだけ？ - 全機能がだめ？ 特定の機能だけ？ - 本番だけ？ ローカルでも同じことが起きる？ - 全員・全機能がだめ → サーバー／データベースの障害（5xx 系）を疑う。特定の機能だけ → その機能のコードやデータを疑う
3. **ログを確認する** - `web_console.log`（§11.10）を見る - Azure のアプリのログ・Gunicorn のログを見る - ブラウザの `F12`（開発者ツール）で、ステータスコードと通信の中身を確かめる
4. **原因を特定する** - ステータスコード（§15）で、4xx（送った側）か 5xx（サーバー）かを切り分ける - 最近のデプロイ・設定の変更がきっかけになっていないかを確かめる
5. **対処する** - 軽い → 設定の直し・データの直し - 重い → 直前の正しく動いていた版へ戻す（ロールバック） - データが壊れた → バックアップ／PITR（§13）から戻す
6. **記録する** - 何が起きて、何をして直したかを、次のためにメモに残す

16.2 症状別・最初に疑うこと

症状	まず疑う	確認場所
画面が真っ白	フロントのビルド忘れ・JSエラー	F12のConsole、`npm run build`したか
「ログインへ」に飛ぶ	セッション切れ・401	ログイン状態、Cookie
POSTが403	CSRF・オリジン設定	`CSRF_TRUSTED_ORIGINS`（§11.9）
400 Bad Request	`ALLOWED_HOSTS`・日付不正	§11.2、§10.3
502 Bad Gateway	Gunicorn起動失敗	Azureログ、環境変数
バックアップが終わらない	スレッド/タスクの失敗	`AsyncTask`の状態、`web_console.log`
時刻が9時間ずれる	UTC/JST混同	§9.6、`USE_TZ`（§11.8）
DBに繋がらない	ファイアウォール・`DB_HOST`	§13.3、§11.6

> ⚠️ **「直前に何を変えたか」を必ず思い出す。**障害の多くは「直近のデプロイ・設定変更」が引き金です。「昨日まで動いていたのに今日ダメ」なら、その間の変更（コミット履歴・設定変更）を真っ先に疑います。`git log` で最近のコミットを確認しましょう。

16.3 ロールバック（前の版に戻す）

深刻な障害で原因がすぐ分からないときは、**まず正常だった版に戻して** 被害を止め、それから落ち着いて原因を調べます。

コピー

```
・コードの問題 → 直前の正常なコミットへ戻して master に push（＝再デプロイ）
・DBの問題     → Azure PITR（§13.3）で正常な時点へ復元
```

> **なぜロールバックを優先するのか？** 障害中はユーザーの業務が止まっています。原因究明を急ぐより、**まず動く状態に戻して業務を再開させる** のが運用の鉄則です。原因調査はその後、再現環境（ローカル）で落ち着いて行います。「止血が先、診断は後」です。

17. 定期メンテナンスとドキュメントのPDF・Web版の再生成

17.1 定期メンテナンスの目安

周期	作業	目的
毎日	ログ確認（`web_console.log`）・エラー有無チェック	異常の早期発見
毎週	重要データのバックアップ取得（§6）・テスト実行	復元準備・品質維持
毎月	古いタスク履歴・操作履歴の整理（§9.6の削除）	DB肥大化防止
随時	依存ライブラリの更新確認・セキュリティ情報の確認	脆弱性対応
デプロイ時	ビルド・collectstatic・動作確認（§14）	安全なリリース

> ⚠️ **履歴データは放置すると無限に増える。**操作履歴（`History`）やタスク履歴（`AsyncTask`）は毎日溜まります。定期的に `delete_History_data`・`delete_AsyncTask_data`（§9.6）で古いものを整理しないと、DBが肥大化して遅くなります。整理の前に必ずバックアップ（§9.6の `generate_*_backup`）を取りましょう。

17.2 ドキュメントのPDF・Web版の再生成手順

この教育資料の原稿は、`kosu/` フォルダの中の `.md`（マークダウン）ファイルです。原稿を書き換えたら、**PDF版**（`kosu/pdf/`）と **Web版**（`kosu/html/`）を作り直します。作り直しは、資料の置き場所（リポジトリ）のいちばん上にある `_build_pdfs.py` という Python のプログラムが、まとめてやってくれます。

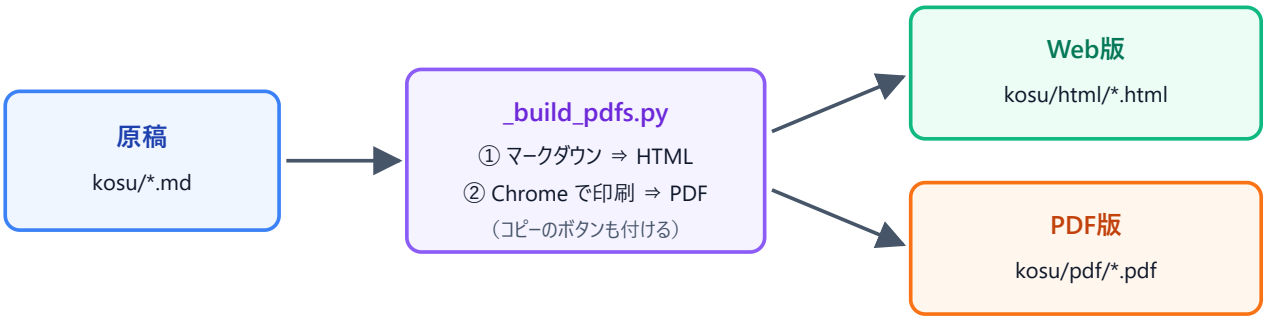


図17-1: 原稿（.md）から、Web版と PDF版がまとめて作られる

準備（最初の 1 回だけ）：

- **Google Chrome**（または Microsoft Edge）が入っていること。PDF は、ブラウザーの「印刷」の働きで作ります。
- **Python** と、次の 3 つの部品（ライブラリ）が入っていること。

> **▼ このコマンドがやること（先に日本語で）**：PDF・Web版づくりに使う部品を、`pip`（Python の部品を入れる道具）で入れます。`markdown` はマークダウンを HTML に変える部品、`pygments` はコードに色を付ける部品、`pikepdf` はできた PDF にコピーのボタンなどを書き足す部品です。

コピー

```
# PDF・Web版づくりに使う 3 つの部品を入れる（最初の 1 回だけ）
pip install markdown pygments pikepdf
```

作り直す（原稿を書き換えるたび）：

> **▼ このコマンドがやること（先に日本語で）**：資料の置き場所のいちばん上のフォルダ（`_build_pdfs.py` がある所）で実行します。`kosu` フォルダの各巻と、`README.md`（この資料の入口）を、Web版と PDF版に作り直します。

コピー

```
# kosu フォルダの各巻と README を、Web版と PDF版に作り直す
python _build_pdfs.py kosu kosu/README.md
```

- `python _build_pdfs.py` … 作り直すプログラムを実行します。
- `kosu` … フォルダの名前を書くと、その中の `.md` を全部作り直します（ただし `README.md` は入らないので、次に別に書きます）。
- `kosu/README.md` … ファイルの名前を書くと、その 1 つだけを作り直します。1 つの巻だけ直した時は、`python _build_pdfs.py kosu/09_運用_保守_デプロイ.md` のように、その巻だけ書くと早く終わります。

▼ こう表示されれば成功:

なぞってコピーすると「…」が別の文字になります。正しくは → コピー

```
Using browser: C:\Program Files\Google\Chrome\Application\chrome.exe
Converting: kosu\00_導入と全体像.md
  -> kosu\pdf\00_導入と全体像.pdf
（…巻の数だけ、くり返し…）
Converting: kosu\README.md
  -> kosu\pdf\README.pdf
Done.
```

`Using browser:` の行は、PDF を作るのに使ったブラウザーです。`Converting:` が「いま作っている原稿」、`->` が「できた PDF」です。最後に `Done.`（終わりました）と出れば完了です。Web版（`kosu/html/` の `.html`）も、同じ時に作り直されています。

⚠️ **PDF を開いたままだと、作り直せないことがある。** Adobe Acrobat で PDF を開いていると、そのファイルが「使用中」になり、上書きに失敗します（`could not write … (is it open in Adobe Acrobat? Close it there and run again)` ＝「書き込めません。Acrobat で開いていませんか？ 閉じてから、もう一度実行してください」と出て止まります）。作り直す前に、その PDF を閉じてください。

💡 **図は、原稿の中に直接書いてある。** この資料の図は、`<svg>`（線や四角を文字で書いて表す絵の書き方）で、`.md` の中に書いてあります。図の文字を直したい時は、`.md` の中の `<text …>` の行を書き換えて、作り直します。

💡 **Web版をインターネットで見られるようにするには:** 資料の置き場所（GitHub のリポジトリ）に、作り直した `kosu/` フォルダをコミットしてプッシュします（やり方は `02_Git_GitHub` を参照）。GitHub Pages（GitHub がファイルを Web ページとして公開してくれる仕組み）で、`https://oltotlo79-rgb.github.io/education/kosu/html/` に出ます。PDF の「コピー」ボタン（Chrome・Edge・VS Code で見た時）は、このページへ飛びます。

18. トラブルシューティング

症状	原因の可能性	対処
`runserver` で `port already in use`	8000番ポートを別プロセスが使用中	既存のサーバーを止めるか、別ポートで起動
バックアップボタンを押しても何も起きない	スレッドが失敗・`AsyncTask` がerror	`web_console.log` と `AsyncTask` の `result` を確認（§10）
ロードで「無効なファイルフォーマットです」	ヘッダーがバックアップ形式と不一致	バックアップで出したExcelをそのまま使う（§8）
ダウンロードしたExcelが空	指定期間にデータが無い	期間を見直す。`validate_dates` の制約も確認（§10.3）
「昨日の日付までしか指定できません」	終了日に今日以降を指定	終了日を昨日以前に（§10.3）
本番でコード変更が反映されない	`master` に push していない／GitHub Actions が失敗した／ブラウザに古い画面が残っている	Actions タブの結果を見る（§14.1）・`git log origin/master`・`Ctrl` + `F5`
デプロイ後に502	Gunicorn起動失敗	環境変数・依存・`wsgi.py` を確認（§14.4）
POSTが403で弾かれる	CSRFオリジン未登録	`CSRF_TRUSTED_ORIGINS` に追加（§11.9）
400 Bad Request（全アクセス）	`ALLOWED_HOSTS` 未登録	本番ドメインを追加（§11.2）
ログが残っていない	1MB×2世代で上書き済み	障害時は早めに `web_console.log` を退避（§11.10）
DBに繋がらない	ファイアウォール・`DB_HOST` 誤り	Azure DBのIP許可・`.env` を確認（§13.3, §11.6）
時刻が9時間ずれる	UTC/JSTの混同	`USE_TZ`・`astimezone` を確認（§9.6, §11.8）
PDF作り直しで `ModuleNotFoundError: No module named 'markdown'`	PDF・Web版づくりの部品が入っていない	`pip install markdown pygments pikepdf`（§17.2）
PDF作り直しで `RuntimeError: could not write … (is it open in Adobe Acrobat? …)`	作り直すPDFを Acrobat などで開いている	そのPDFを閉じてから、もう一度実行（§17.2）

症状	原因の可能性	対処
マイグレーションで本番データが消えた	列削除マイグレーションを適用	PITR/バックアップから復元。今後は §14.3を厳守

---

19. 演習問題

理解度を確認しましょう。答えは本文中にあります。

問1. バックアップ機能は「非同期」で動きます。ユーザーがボタンを押してからExcelをダウンロードするまでの流れを、`backup` → `check_task_status` → `download_file` の3関数の役割に触れながら、自分の言葉で説明してください。（§5・§10）

問2. `generate_kosu_backup`（§6）で、`str(item.name)` のように `name` だけ `str()` で囲んでいるのはなぜですか。

問3. ロード処理の「置き換え方」には4種類（上書き・スキップ・全置換・追加だけ）あります。それぞれどの表が該当し、どういう挙動か説明してください。（§8・§9.5）

問4. `validate_dates`（§10.3）は、終了日に「今日以降」を許しません。なぜそのような制約があるのか、業務的な理由を答えてください。

問5. `download_file`（§10.5）は、ファイルを返したあと3秒待ってから削除します。「すぐ削除」でも「削除しない」でもなく「3秒後に削除」にしている理由を、2つの観点（ディスク／ダウンロード中）から説明してください。

問6. `settings.py` で本番URLを変更するとき、更新が必要な3か所を挙げてください。（§11.9）

問7. `RotatingFileHandler` の `maxBytes` と `backupCount` の意味を説明し、本アプリの設定（1MB・1世代）だとログが最大何MBになるか答えてください。（§11.10）

問8. `wsgi.py` の `application` という変数は、本番で何（どのソフト）に・どう使われますか。（§12・§13）

問9. 本番マイグレーションで「列を削除する」のが特に危険な理由と、事故を防ぐための鉄則を3つ挙げてください。（§14.3）

問10. ユーザーから「POSTすると403になる」と連絡がありました。まず疑うべき設定はどれですか。また「画面が真っ白」ならまず何を疑いますか。（§15・§16.2）

問11. アプリ内バックアップ（§6）とAzureのPITR（§13.3）は、どちらも「バックアップ」ですが役割が違います。それぞれ「誰が・何を・どう戻すか」を対比して説明してください。

問12. django-q（`Q_CLUSTER`）が設定されているのに、バックアップ処理は実際には `threading` で動いています。この事実が運用上どんな意味を持つか、`qcluster` 起動の要否に触れて説明してください。（§3.4・§10.2）

---

20. この章のまとめ

この章では、本アプリを **動かし続け・直し・本番へ出す** ための知識を、実コードを1行ずつ読みながら学びました。

- **3つの環境** — ローカル（実験）・ステージング（確認）・本番（公開）を分け、設定（`.env`）で差を吸収する。本アプリは実態としてローカルと本番の2環境。
- **日常運用コマンド** — `runserver`・`migrate`・`makemigrations`・`qcluster`・`collectstatic`・`npm build`・テスト。それぞれの役割と使うタイミングを覚えた。
- **バックアップ／復元**（`tasks.py`） — openpyxlでExcelを「ブック→シート→セル」で作り、pandasで重複を消し、Excel→DBへ流し込む。ロードの置き換え方は表ごとに4種類（上書き・スキップ・全置換・追加だけ）。
- **非同期フロー**（`asynchronous_views.py`） — タスクID発行→別スレッド起動→ポーリング（202 pending）→ダウンロード→3秒後に自動削除、という一連を1行ずつ追った。
- **設定**（`settings.py`） — `ALLOWED_HOSTS`・`CSRF_TRUSTED_ORIGINS`・`DATABASES`・`Q_CLUSTER`・WhiteNoise・`MEDIA_ROOT`・`LOGGING`（RotatingFileHandler）の意味を理解した。本番URL変更時は3か所更新。
- `wsgi.py` — Webサーバー（Gunicorn）とDjangoをつなぐ玄関。`application` が本番の入口。
- **本番アーキテクチャ** — Azure App Service + Gunicorn + WhiteNoise + Azure Database for PostgreSQL フレキシブルサーバー（接続・ファイアウォール・TLS・自動バックアップ・PITR）。
- **デプロイ** — `master` に push すると、GitHub Actions が画面のビルド・collectstatic・Azure への送り出しまで自動で行う。毎回の手順は ①テスト ②手元でだけ書き換えた 2 か所を戻す ③コミット ④ `git push origin master` ⑤Actions が緑か見る ⑥（モデルを変えた時だけ）本番の migrate ⑦本番で確認。本番マイグレーションは5つの鉄則を厳守（やり方は前任者に確かめる）。
- **HTTPステータスコード** — 4xx（送り手の問題）と5xx（サーバーの問題）で原因の方向を切り分ける。
- **障害対応** — 症状把握→切り分け→ログ確認→原因特定→対処（深刻ならロールバック）→記録。「止血が先、診断は後」。
- **定期メンテ・PDF再生成** — 履歴の定期整理でDB肥大化を防ぎ、`python _build_pdfs.py kosu kosu/README.md` で資料をPDF・Web版に作り直す。

運用は「地味だが最も実務で使う」分野です。この章を**辞書**として手元に置き、困ったら該当節を引いてください。

次は最終章です。本資料で登場したすべての専門用語を、五十音順に引ける一覧にまとめます。

→ **次章:** 10_用語集.md — 全用語の五十音順リファレンス